<u>1c2025</u>

1era Fecha

1.

a. Describir el modelo OSI y explicar objetivos y capacidades de cada capa.
b. Explicar que es un socket y que modelo de concurrencia usan.

Sol:

a) La idea del modelo OSI (Open System Interconnect), es dividir el problema de "comunicar dos aplicaciones" en capas, cada una con responsabilidades acotadas y bien definidas. Cada capa **usa** los servicios de la de abajo y **ofrece** servicios a la de arriba. La idea de separar en capas independientes es para que estas se puedan cambiar (cumpliendo el mismo objetivo pero distinto procedimiento) y todo siga normal, también porque un software es muy complejo y es bueno poder separar responsabilidades. Tenemos 7 capas:

Capa 1 - Física

- **Objetivo:** Enviar bits por el medio físico.
- **Capacidades:** Define voltajes, conectores, frecuencias y tipos de señales.

Capa 2 - Enlace de Datos (Data Link):

- **Objetivo:** Asegurar la transmisión libre de errores entre dos nodos conectados físicamente.
- **Capacidades:** Organiza los bits en tramas (frames) y maneja la detección de errores (CRC), control de acceso al medio (MAC) y direccionamiento físico (MAC).

Capa 3 - Red (Network):

- **Objetivo:** Determinar la ruta que seguirán los datos hasta su destino, posiblemente atravesando varias redes.
- **Capacidades:** Direccionamiento lógico (IP), enrutamiento, fragmentación, decisiones de ruta.

Capa 4 - Transporte (Transport):

- **Objetivo:** Dar una comunicación extremo a extremo entre procesos.
- **Capacidades:** Segmentación y reensamblado, control de flujo, manejo de errores, entrega ordenada y confiable (TCP) o no confiable pero liviana (UDP).

Capa 5 - Sesion (Session):

- **Objetivo:** Administrar el diálogo entre aplicaciones.
- **Capacidades:** inicio, mantenimiento y finalización de sesiones, checkpoints, recuperación.

Capa 6 - Presentación (Presentation):

- **Objetivo:** Que ambos extremos interpreten los datos igual ("traduce").
- **Capacidades:** Conversión de formatos, compresión, encriptación, negociación de codificaciones.

Capa 7 - Aplicación (Application):

- **Objetivo:** Proporcionar servicios de red directamente a las aplicaciones del usuario.
- **Capacidades:** Protocolos como HTTP, FTP, SMTP, etc., interfaz directa entre el software de usuario y la red.

b) Un **socket** es una abstracción que da el sistema operativo para enviar/recibir datos entre procesos a través de la red. Se identifica por (IP origen, puerto origen, IP destino, puerto destino, protocolo).

Un socket no impone ningún modelo de concurrencia, pero las implementaciones típicas de servidores utilizan modelos como thread-per-connection, thread pool o I/O asíncrono con event loop que son aplicados a sockets, todos estudiados en la materia.

2. Describir y comparar los efectos negativos que tiene en los programas concurrentes los:

- deadlocks
- race conditions
- starvation
- busy wait

Sol:

Safety: Exclusión mútua, dos procesos no deben intercalar (sub)secuencias de instrucciones, por ej, incremento de una variable global.

Liveness: Ausencia de starvation ⇒ todo proceso que esté listo para recibir un recurso, debe recibirlo en algún momento.

Deadlock

- Situación en la que un conjunto de procesos queda bloqueado indefinidamente porque cada uno espera un recurso que otro del conjunto tiene, es decir, A espera a 2 teniendo 1 y B espera a 1 teniendo a 2.
- Efecto negativo: **liveness** se rompe; esos procesos no progresan nunca. Si son críticos, el sistema parece “congelado”.

Race Condition (Condición de carrera)

- Dos o más procesos acceden a estado compartido sin sincronización adecuada y el resultado depende del orden de ejecución.
- Efecto negativo: bug intermitente, difícil de reproducir; errores de lógica, datos corruptos.

Starvation (inanición)

- Un proceso siempre es “postergado” y nunca obtiene el recurso, aunque el sistema en conjunto sigue funcionando.
- Efecto negativo: injusticia / falta de *fairness*; viola propiedades de progreso para ese proceso en particular. Los algoritmos de prevención como wait-die y wound-wait reducen este problema al ordenar las transacciones por antigüedad y asegurar progreso.

Busy wait (espera activa)

- Un hilo se queda en un loop preguntando continuamente por una condición, consumiendo CPU.
- Efecto negativo: mal uso de CPU, puede degradar a los demás procesos aunque lógicamente la sincronización sea correcta. Se prefiere bloquearse (semáforos, condition variables) y liberar el procesador.

3.

a. Explicar el algoritmo de detección de deadlocks distribuido en programación concurrente distribuida.
b. Diagramar un ejemplo de detección de deadlock y explicarlo paso a paso.
c. Explicar cómo se puede evitar esto (osea prevención de deadlocks - wait and die y wound wait).

sol:

3. Detección de deadlocks distribuida + ejemplo + prevención (wait-die / wound-wait)

a) Algoritmo de detección de deadlock distribuido

En ambiente distribuido no hay memoria compartida global; la detección se hace con **mensajes**. El apunte describe:

- Cada proceso mantiene info de **quién espera qué recurso de quién** (un fragmento del grafo de espera).
- Los pedidos de recursos se propagan como mensajes a lo largo de esa cadena de dependencias.
- Si una petición "vuelve" al proceso que la originó, se detectó un **ciclo** ⇒ hay deadlock. Concurrentes

En palabras: *"si yo necesito un lock que lo tiene un proceso que a su vez me manda a mí que necesita un lock que tengo yo, ahí veo la dependencia circular y detecto el deadlock"*.

b) Ejemplo paso a paso con diagrama

Supongamos tres procesos distribuidos:

- **P1** en nodo A, con lock `L1`.
- **P2** en nodo B, con lock `L2`.
- **P3** en nodo C, con lock `L3`.

Situación de espera:

- P1 tiene `L1` y pide `L2` → P1 espera de P2.
- P2 tiene `L2` y pide `L3` → P2 espera de P3.
- P3 tiene `L3` y pide `L1` → P3 espera de P1.

Flujo (estilo examen):

1. P1 no puede obtener `L2`, envía mensaje de **petición** hacia P2 indicando "P1 espera L2 de P2".
2. P2, al recibirlo, ve que él está esperando `L3` de P3 → reenvía el mensaje a P3 agregando su parte ("P2

2. P2, al recibirlo, ve que él está esperando `L3` de P3 → reenvía el mensaje a P3 agregando su parte ("P2 espera L3 de P3").
3. P3 recibe la cadena y ve que está esperando `L1` de P1 → reenvía la petición hacia P1.
4. P1 recibe un mensaje que menciona a P1 nuevamente en la cadena (P1 → P2 → P3 → **P1**).
5. Como el mensaje "volvió", P1 concluye que existe un **ciclo en el grafo de espera** ⇒ hay deadlock.
6. Alguno de los procesos (según política) debe abortarse/liberar recursos para romper el ciclo.

En un gráfico tipo examen dibujás los tres nodos y flechas P1→P2→P3→P1.

c) Prevención de deadlocks: wait-die y wound-wait

Ambos esquemas usan **timestamps** (antigüedad de la transacción/proceso). Concurrentes

- Cada proceso tiene un timestamp `TS(P)` (más chico = más viejo).

Wait-Die

- Si un proceso **viejo** pide un recurso tomado por uno **más joven**: se le permite **esperar** (wait).
- Si un proceso **joven** pide un recurso tomado por uno **más viejo**: el joven **muere** (abort), libera recursos y reintenta más tarde.

Se llama "wait-die" porque solo el viejo espera; el joven "muere".
Con esto nunca tenés ciclos de espera de jóvenes sobre viejos → no hay deadlock.

Wound-Wait

- Si un proceso **viejo** pide un recurso tomado por uno **más joven**: el viejo lo "hiere" (wound): fuerza a abortar al joven, que libera el recurso inmediatamente.
- Si un proceso **joven** pide un recurso tomado por uno **más viejo**: el joven **espera**.

Aquí el viejo nunca espera a un joven. Nuevamente, se ordenan las posibles esperas por antigüedad y se evita el ciclo.

En ambos casos se penaliza al que "menos trabajo acumulado" tiene (el más joven), reduciendo re-trabajo.

4.

a. Enumerar posibles eventos en un ambiente distribuido.
b. Explicar cómo se calcula el costo y complejidad en un ambiente distribuido y comparar el cómputo que hay que hacer en un sistema centralizado.

5. Se tiene un sistema de compra de pasajes de micros a larga distancia. Un usuario selecciona una fecha origen y destino, y el sistema le muestra los asientos disponibles en el micro. El usuario selecciona uno y luego pasa a la fase de pagos.

a. Modelar un sistema de transaccionalidad para este problema y explicar cómo se compone qué mensajes se envían y como puede ayudar a prevenir problemas (por ej que se reserve un asiento sin pagarlo).
b. Diagramar o escribir las entidades y mensajes que se envían y un flujo general del sistema.
c. Explicar dos casos de éxito y dos casos de falla y cómo se comporta el sistema frente a estos.

Como condición de aprobación, el **punto 5** debe estar bien

4. Eventos en ambiente distribuido + costo y complejidad vs centralizado

a) Posibles eventos en un ambiente distribuido

Según el apunte de "Ambientes Distribuidos", tenés entidades con estado interno y **eventos externos** que disparan acciones. Concurrentes

Ejemplos de eventos en un sistema distribuido:

- **Recepción de un mensaje** (request, respuesta, ACK, timeout).
- **Envió exitoso o fallo de envío**.
- **Timeouts** (no llegó un mensaje antes de cierto Δ).
- **Acciones locales**: fin de una computación local, cambio de estado interno.
- **Entrada/salida externa**: pedido de un usuario, llegada de datos de sensores, etc.
- **Eventos de falla**: caída de nodo, caída de enlace, recuperación de nodo, elección de nuevo líder, etc.

Cada combinación *(estado de la entidad, evento externo)* dispara una **regla** de comportamiento.

b) Costo y complejidad en ambiente distribuido

En distribuidos el costo no es solo "operaciones de CPU", también:

- **Número de mensajes** enviados.
- **Tamaño total de datos** transmitidos.
- **Cantidad de hops** (enlaces) por los que pasa cada mensaje.
- **Tiempo total** considerando latencias de red y límites Δ mencionados en el apunte. Concurrentes

Entonces, para un algoritmo distribuido solemos medir:

- Complejidad en **mensajes**: por ejemplo, O(N) mensajes para elección de líder.
- Complejidad en **tiempo distribuido**: cantidad de rondas de comunicación hasta terminar.

Comparación con sistema centralizado

- En un sistema centralizado la complejidad se mide casi solo en **operaciones de CPU y accesos a**

- En un sistema centralizado la complejidad se mide casi solo en **operaciones de CPU y accesos a memoria**, porque todo está en la misma máquina.
- En cambio, en el distribuido muchas veces el costo dominante es la **comunicación**, que es varios órdenes de magnitud más lenta que la memoria local.
- Por eso un algoritmo que es barato en CPU pero manda muchos mensajes puede ser peor que otro más "pesado" localmente pero con menos tráfico.

En una respuesta de final, aclarar que el diseño cambia: se busca minimizar mensajes, latencia y tamaño de logs distribuidos.

---

5. Sistema de compra de pasajes: modelo transaccional

(punto clave para aprobar)

Te piden:

1. Modelar sistema transaccional.
2. Entidades, mensajes y flujo.
3. Dos casos de éxito y dos de falla.

Todo esto encaja perfecto con la parte de **Transacciones** y **Control de concurrencia** del apunte.

Concurrentes

a) Entidades del sistema

Podés plantear algo así:

- **Cliente (navegador/app)**
- **Servicio de Pasajes** (backend / API).
- **Servicio de Pagos** (pasarela de pago externa).
- **Base de Datos Transaccional** con tablas:

- **Base de Datos Transaccional** con tablas:
  - `Viaje(id, fecha, origen, destino, …)`
  - `Asiento(id_viaje, nro_asiento, estado, …)` con estados: `LIBRE`, `RESERVADO`, `VENDIDO`.
  - `Reserva(id_reserva, id_viaje, asiento, estado, timestamp, id_usuario, …)`
- Eventualmente un **coordinador de transacciones** si lo querés hacer más distribuido (2PC).

b) Modelo transaccional (qué garantiza)

Usamos transacciones con propiedades **ACID**:

- **Atomicidad:** o se completa la reserva + pago + marcado de asiento vendido, o nada de eso queda aplicado.
- **Consistencia:** nunca hay dos ventas para el mismo asiento.
- **Aislamiento:** si dos usuarios compiten por el mismo asiento, uno solo lo ve finalmente como suyo.
- **Durabilidad:** una vez que se confirma el pago, el asiento queda vendido aunque se caiga el sistema después.

Podés mencionar que se implementa como en el apunte: cada transacción trabaja con una copia privada y al `COMMIT` se aplican los cambios al storage estable, manteniendo un log para recuperación. Concurrentes

c) Flujo general y mensajes

Dividí en dos transacciones:

1) Consulta de disponibilidad (solo lectura)

1. Cliente envía: `BuscarViajes(fecha, origen, destino)`.
2. Servicio de Pasajes abre **Transacción T1 (read-only)**:
   - Lee filas de `Viaje` y `Asiento` con estado `LIBRE`.
3. Devuelve al cliente listado de viajes y asientos disponibles.
4. T1 hace `COMMIT` (no modifica nada).

2) Reserva + Pago (transacción principal)

2) Reserva + Pago (transacción principal)

Para simplificar podés hacer una **transacción larga** coordinando reserva y pago, o dos transacciones encadenadas con un estado intermedio `RESERVADO`. Una solución razonable para el final:

Paso 1: iniciar reserva

- Cliente elige `viaje X, asiento Y` y manda `ReservarAsiento(X, Y)`.
- Servicio de Pasajes abre **Transacción T2**:
  1. `SELECT ... FOR UPDATE` del asiento `(X,Y)` → toma lock.
  2. Verifica que el asiento esté `LIBRE`.
  3. Lo pasa a estado `RESERVADO` y crea una fila en `Reserva` con estado `PENDIENTE_PAGO` y un timestamp de expiración.
  4. `COMMIT T2`.

Resultado: el asiento queda "bloqueado" para otros usuarios durante un tiempo.

Paso 2: pago

- Cliente pasa a pantalla de pago y manda `PagarReserva(id_reserva, datos_tarjeta)`.
- Servicio de Pasajes inicia **Transacción T3**:
  1. Lee la fila de `Reserva` y del `Asiento` correspondiente con lock.
  2. Verifica:
     - que la reserva esté `PENDIENTE_PAGO`,
     - que no haya expirado.
  3. Llama al **Servicio de Pagos** (mensaje `AutorizarPago(monto, tarjeta, id_reserva)`).
     - Si la pasarela responde OK, marca la reserva como `CONFIRMADA` y el asiento como `VENDIDO`.
     - Si responde error, marca la reserva como `RECHAZADA` y si corresponde vuelve el asiento a `LIBRE`.
  4. `COMMIT T3`.

Otros usuarios que intenten reservar el mismo asiento se van a encontrar con el lock/estado `RESERVADO` y deberán reintentar otro asiento.

d) Cómo evita problemas (asiento reservado sin pagar, etc.)

- El estado `RESERVADO` tiene **timeout**: si no se paga antes de cierto tiempo, un proceso de limpieza corre otra transacción que expira reservas pendientes y vuelve los asientos a `LIBRE`.
- El uso de locks con **dos fases** (2PL) o timestamp-ordering asegura que dos transacciones que compiten por el mismo asiento no lo marquen como vendido simultáneamente. Concurrentes
- Podés mencionar que si varias reservas involucran múltiples asientos (ej. comprar 4 asientos juntos), ahí pueden aparecer deadlocks y se puede aplicar **wait-die o wound-wait** sobre las transacciones para prevenirlos.

e) Casos de éxito

Éxito 1 – Compra normal

1. Usuario A consulta disponibilidad y ve que el asiento 10 está libre.
2. Envía `ReservarAsiento`. T2 bloquea el asiento, lo marca `RESERVADO`, crea `Reserva(PENDIENTE_PAGO)`, commit.
3. Usuario A paga. T3 verifica reserva, llama a pasarela, recibe "aprobado", marca `VENDIDO` + `CONFIRMADA`, commit.
4. Si otro usuario consulta después, ya ve ese asiento como vendido.

Éxito 2 – Usuario cambia de asiento antes de pagar

1. A reserva asiento 10 → `RESERVADO`.
2. A decide cambiar al asiento 12:
   - Nueva T2' marca asiento 12 como `RESERVADO` y cancela la reserva anterior (o la deja expirar).
3. Paga la nueva reserva; T3' vende asiento 12.
4. Proceso de limpieza luego libera el asiento 10 s edó pendiente.

f) Casos de falla y comportamiento del sistema

Falla 1 – Dos usuarios eligen el mismo asiento casi al mismo tiempo

1. Usuario A y B eligen asiento 10.
2. T2 de A llega primero: toma lock, ve `LIBRE`, pasa a `RESERVADO`, commit.
3. T2 de B cuando intenta `SELECT ... FOR UPDATE` queda bloqueada o al leer ve que el asiento está `RESERVADO` / `VENDIDO`.
4. B recibe respuesta de "asiento no disponible, elija otro".
   - No hay doble venta, solo se rechaza la operación de B.

Falla 2 – Pago rechazado / caída de pasarela

1. A reserva asiento 10 (T2 ok).
2. En T3, el Servicio de Pagos responde "rechazado" o hay timeout.
3. Política del sistema:
   - Marca la reserva como `RECHAZADA`.
   - Puede pasar el asiento a `LIBRE` inmediatamente o dejarlo `RESERVADO` un rato hasta confirmar con el usuario.
4. En cualquier caso, **no se marca como** `VENDIDO` → nadie tiene un pasaje inválido.

Falla 3 – Cliente se va sin pagar (abandona la web)

1. A reserva asiento 10; nunca llega el mensaje `PagarReserva`.
2. Un proceso periódico ejecuta una transacción de **limpieza**:
   - Busca reservas `PENDIENTE_PAGO` con timestamp vencido.
   - Cambia `RESERVA` a `EXPIRADA` y asiento a `LIBRE`.
3. El asiento vuelve a estar disponible para otros usuarios; la reserva abandonada no genera venta fantasma.

Falla 4 – Caída del servidor en medio de una transacción

- Gracias al **log de transacciones**:

Falla 4 – Caída del servidor en medio de una transacción

- Gracias al **log de transacciones**:
  - Si T3 no llegó a `COMMIT`, al reiniciar se hace *rollback* de esa transacción y el asiento vuelve al estado anterior (por ejemplo `RESERVADO` pendiente de reintento o `LIBRE`).
  - Si el commit quedó registrado, el asiento ya está `VENDIDO` y la venta se considera cerrada (durabilidad).

Con estos casos mostrás claramente que:

- El sistema evita doble venta.
- No deja asientos "fantasma" sin pagar por tiempo indefinido.
- Tolera fallas de pago, abandono de usuario y caídas del servidor.

📌 3) Deadlocks en concurrencia distribuida

✔ Algoritmo de detección de deadlocks distribuido

En sistemas distribuidos **no existe memoria global**, por lo tanto el grafo de espera no puede construirse en un único nodo. La detección se hace **por mensajes entre procesos**:

1. Cuando un proceso Pi espera un recurso que posee Pj, envía un mensaje **Request → Pj**.
2. Si Pj también está bloqueado esperando a Pk, reenvía el mensaje indicando la dependencia.
3. Si el mensaje regresa al proceso que lo originó, se detecta **un ciclo**, lo cual implica **deadlock**.

> Si el mensaje vuelve al origen, existe una dependencia circular → Deadlock.

📌 **Condición de detección distribuida:**

$$Pi \to Pj \to \cdots \to Pi$$

📌 Ejemplo paso a paso con diagrama

Proceso	Tiene	Solicita
P1	Recurso A	Recurso B
P2	Recurso B	Recurso C
P3	Recurso C	Recurso A

Mensajes de detección:

```yaml
P1 → P2: "Espero por B"
P2 → P3: "P1 y yo esperamos por C"
P3 → P1: "P1-P2-P3"
```

📌 Como el mensaje vuelve a **P1**, se detecta un ciclo ⇒ **Deadlock distribuido**.

Diagrama:

```nginx
P1 → P2 → P3 → P1   ⇒   ciclo ⇒ deadlock
```

🔐 Prevención de Deadlocks: Wait–Die y Wound–Wait

Ambos usan **timestamps**: menor TS = más viejo.

🕒 Wait–Die (viejo espera, joven muere)

- Si **viejo pide recurso** tomado por **joven** → **espera**.
- Si **joven pide recurso** tomado por **viejo** → **se aborta (muere)** y reintenta luego.

> Los jóvenes nunca esperan a viejos ⇒ no hay ciclos.

🩸 Wound–Wait (viejo hiere)

- Si **viejo pide a joven** → el **viejo lo aborta al joven (lo hiere)**.
- Si **joven pide a viejo** → el joven **espera**.

> El viejo siempre avanza → no se generan ciclos.

📌 4) Eventos y costo en ambientes distribuidos

✔ Enumeración de eventos en un sistema distribuido

- Recepción de mensajes (REQUEST, ACK, RELEASE, VOTE, COMMIT).
- Timeouts por falta de respuesta.
- Fallas de nodo (crash) y recuperación.
- Mensajes de elección de líder o token.
- Mensajes transaccionales (PREPARE, ABORT, COMMIT).
- Ejecución local de reglas según eventos.

📌 Los procesos reaccionan a **eventos externos**, no a un estado global compartido.

📌 Costo y complejidad en sistemas distribuidos

El costo computacional se mide principalmente en:

- **Número de mensajes**
- **Saltos (hops)** entre nodos
- **Latencia de red**
- **Tolerancia a fallas**

En cambio, en sistemas centralizados el costo depende de:

- Tiempo de CPU
- Memoria
- Acceso directo a disco/memoria

📌 **Comparación:**

📌 **Comparación:**

Centralizado	Distribuido
CPU / memoria	Mensajes, latencia, fallas
Acceso local	No hay estado global
Menor costo	Costo alto por coordinación

📌 5) Sistema transaccional de compra de pasajes (crucial)

🎯 Objetivo

Evitar:

- Venta duplicada del mismo asiento.
- Asientos reservados sin pago confirmado.

✔ Modelo de estados de un asiento

```bash
LIBRE → RESERVADO → VENDIDO
          ↓
      (timeout)
          ↓
       LIBRE
```

- **RESERVADO** debe tener *tiempo límite*; si no hay pago, vuelve a **LIBRE**.

✔ Transacciones del sistema

🔐 T1: Reserva de asiento

- Lock de escritura en el asiento.
- Si está LIBRE ⇒ pasa a **RESERVADO**.
- Se crea registro en tabla **RESERVA (PENDIENTE_PAGO)**.
- **Commit.** No hay pago todavía.

> Garantiza que mientras el usuario paga, nadie más reserva el mismo asiento.

💰 T2: Pago

- Lee RESERVA y valida estado.
- Solicita cargo a sistema de pagos.
- Si pago aprobado ⇒ asiento pasa a **VENDIDO**.
- Si pago rechazado ⇒ reserva **RECHAZADA** y asiento vuelve a **LIBRE**.
- **Commit.**

🕒 Proceso de Timeout

- Cancela reservas no pagadas y devuelve asiento a **LIBRE**.
- Evita “asientos congelados eternamente”.

💬 Mensajes entre entidades

Mensaje	Fuente	Destino	Acción
BuscarViajes	Cliente	Pasajes	Read
ReservarAsiento(X,Y)	Cliente	Pasajes	T1
PagarReserva(ID)	Cliente	Pagos/Pasajes	T2
ConfirmarPago	Pagos	Pasajes	VENDIDO
CancelarReserva	Timeout	Pasajes	LIBRE

🟢 Casos de Éxito

A) Compra normal

1. Cliente reserva asiento.
2. T1 lo marca como **RESERVADO**.
3. Paga.
4. T2 confirma y pasa a **VENDIDO**.

4. T2 confirma y pasa a **VENDIDO**.

B) Cambio de asiento

1. Cliente reserva 24.
2. Cancela/expira.
3. Reserva 15.
4. Paga ⇒ solo el 15 queda **VENDIDO**.

🔴 Casos de Falla

Falla 1: Pago rechazado

- T2 devuelve estado a **LIBRE**.
- No queda asiento bloqueado.

Falla 2: Cliente abandona

- Timeout devuelve **RESERVADO → LIBRE**.
- No quedan asientos ocupados sin pagar.

📌 Conclusión escrita del punto

> El sistema usa dos transacciones separadas: una para reserva y otra para pago. La reserva bloquea temporalmente el asiento y evita doble asignación, y el pago confirma la venta. El estado intermedio "RESERVADO" posee timeout, asegurando que no queden lugares bloqueados sin abonar. Se respetan ACID, se evita doble venta y se garantiza consistencia aún ante falla

2da Fecha

1. Fork-join:
Definición, definir work stealing y explicar por qué es mejor implementar work stealing a que tener una sola cola de tareas compartida.

2. Redes de petri:

a. definición y para qué sirve. Explicar la diferencia entre ordinaria vs general.

b. Definición de grafo de alcance y ejemplo
c. graficar una red de petri de productor-consumidor con buffer acotado

3. Actores:
Explicar cuál es la motivación del modelo, características detalladas de la entidad actor y ciclo de vida de un actor de actix.

4. Entidades:
Qué son, capacidades y restricciones temporales.

5. Control de concurrencia:
Definir 2 phase locking, timestamps y concurrencia optimista.

Situación de supermercado con mucha contención en hora pico, ventajas y desventajas en esa situación para los 3 tipos de control, elegir uno y explicar por qué es el mejor para la situación.

📌 1) Fork–Join

✔ Definición

Fork–Join es un modelo de paralelismo donde **un problema se divide recursivamente en subtareas (fork)**, las cuales se ejecutan en paralelo, y luego **sus resultados se combinan (join)**. Es muy usado en algoritmos paralelos divide-and-conquer (quicksort, mergesort paralelo, tareas independientes).

✔ Work Stealing

Cada CPU (worker thread) posee **su propia cola de tareas**. Cuando un worker termina lo que tiene, **"roba" trabajo de otras colas**, usualmente del final (bottom) de otra cola.

✔ Por qué es mejor que una sola cola compartida

- Una cola compartida requiere **bloqueos o acceso sincronizado**, generando contención.
- Work stealing **reduce colisiones**, porque la mayor parte del tiempo cada CPU consume su propia cola **sin lock**.
- El bloqueo ocurre solo cuando un worker está idle y necesita robar, pero eso sucede **mucho menos** que acceder a una única cola global.

📌 **Conclusión:** Work stealing escalabiliza mejor y aumenta el throughput bajo carga.

📌 2) Redes de Petri

✔ Definición y para qué sirven

Una red de Petri es un formalismo matemático para modelar **sistemas concurrentes, distribuidos, asíncronos y no deterministas**. Permite analizar **paralelismo, sincronización, conflictos, bloqueos, reachability**.

Se compone de:

- **Lugares (circulos)**: condiciones/recursos.
- **Transiciones (rectángulos/barra)**: acciones.
- **Fichas (tokens)**: estado del sistema.

✔ Petri Ordinaria vs General

Tipo	Característica
Ordinaria	Arcos con peso 1; requiere 1 ficha para disparar
General	Arcos pueden tener pesos > 1, requieren múltiples fichas

✔ Grafo de alcance (Reachability Graph)

Es el **grafo dirigido que muestra todos los estados alcanzables** desde un marcado inicial, donde los nodos son marcados y las aristas corresponden a transiciones disparadas.

Ejemplo simple
Marcado inicial: un productor puede producir si hay espacio, un consumidor puede consumir si hay producto.

Estados posibles podrían representarse como:
$(buffer = 0)$, $(buffer = 1)$, $(buffer = 2)$ … con transiciones P (produce) y C (consume).

✔ Red de Petri para Productor–Consumidor con buffer acotado

```scss
(Espacio) ●●●  →  [Producir] → (Productos) ○○
                  [Consumir] ←
```

Representación formal:

- **Lugares**:
  `BufferLibre`, `BufferOcupado`, `ProductorListo`, `ConsumidorListo`
- **Transiciones**: `Produce`, `Consume`
- **Tokens**:
  Cantidad de espacios libres, items en buffer, productor listo, consumidor listo.

Buffer acotado: `BufferLibre` tiene *N* fichas iniciales.

📌 3) Modelo de Actores

✔ Motivación

Resolver concurrencia **sin memoria compartida**, evitando race conditions y necesidad de locks, usando **comunicación por mensajes**, aislamiento y no compartición de estado.

✔ Características de un Actor

- Posee **estado privado** no accesible externamente.
- Procesa **mensajes/asíncronos** uno a la vez.
- Puede:
  1. Enviar mensajes a otros actores.
  2. Crear nuevos actores.
  3. Cambiar su propio estado interno.
- La comunicación es **FIFO** por buzón (mailbox).
- No hay memoria compartida → **sin locks**, sin races.

✔ Ciclo de vida en Actix (Rust)

1. **Spawning** (creación del actor).
2. **Inicio** (`started()`).
3. **Recepción de mensajes** (`handle()` o `StreamHandler`), con auto–schedulling.
4. **Cambio de estado interno según mensajes**.
5. **Parada voluntaria o falla** (`stopping()`, `stopped()`).
6. **Liberación de recursos.**

6. **Liberación de recursos.**

Actix garantiza **un mensaje a la vez** por actor.

📌 4) Entidades en ambientes distribuidos

✔ Definición

Son **procesos o agentes que reaccionan a eventos externos** intercambiando mensajes. No comparten memoria, por lo tanto **no existe estado global accesible**, solo estados locales.

✔ Capacidades

- Enviar/recibir mensajes.
- Ejecutar acciones locales.
- Mantener estado.
- Participar en protocolos (elección, commit, exclusión mutua, etc.).

✔ Restricciones temporales

- No puede leer estado global → solo **reacciona a eventos**.
- Las acciones ocurren en **función de mensajes o timeouts**.
- El tiempo y orden de mensajes **no es determinista**, depende de la red.

📌 5) Control de concurrencia

✔ Definición breve de cada esquema

🔐 Two Phase Locking (2PL)

- Usa **locks de lectura/escritura**.
- Dos fases:
  - Crecimiento: se toman locks.
  - Decrecimiento: se liberan.
- Garantiza **serializabilidad**, pero puede **causar deadlocks**.

⏱ Timestamp Ordering

- Cada transacción tiene un **timestamp (TS)**.
- El orden de acceso a elementos respeta los TS.
- Conflictos se resuelven **abortando al más joven o inconsistentes**.
- Sin bloqueos → **sin deadlock**, pero hay abortos.

💡 Concurrencia Optimista

- No usa locks al modificar.
- Cada transacción trabaja en **copia privada**.
- Al momento del commit se **valida** si hubo conflicto:
  - Si sí → rollback.
  - Si no → commit.
- Ideal cuando los conflictos son raros.

🛒 Situación: Supermercado con mucha contención (hora pico)

Muchos clientes compiten por productos/stock → **alta contención**.

Método	Ventaja	Desventaja
2PL	Garantiza consistencia	Contención muy alta, deadlocks, demora
Timestamps	Sin deadlock	Muchos abortos por reintentos
Optimista	Sin bloqueos mientras se compra	**Muchísimos abortos** en hora pico, baja performance

📌 **Conclusión:** El mejor es **2PL**, porque:

> Aunque es más lento, evita conflictos constantes en ambiente de altísima contención. Optimista y timestamps desperdician recursos abortando repetidamente.

📌 Un sistema donde muchos compiten → conviene **bloquear y asegurar** en lugar de rehacer muchas veces.

Ejercicio 5 - Redes de Petri

Un sistema de préstamo de bicicletas urbanas opera bajo las siguientes reglas:

1. La estación tiene capacidad para 5 bicicletas.
2. Los usuarios pueden retirar bicicletas solo si hay al menos una disponible.
3. Un camión de reposición agrega 3 bicicletas a la estación cada vez que queda vacía.
4. El camión no puede reponer si aún hay bicicletas en la estación.
5. Las acciones de retiro y reposición son atómicas.

Consignas:

5.1) Modelar este sistema con una Red de Petri, incluyendo:

- Lugares (con marcas iniciales)
- Transiciones y arcos.

Las anotaciones y el diagrama deben ser claros (ej: "Retirar", "Reponer").

5.2) Explicar cómo el modelo desarrollado garantiza:

1. Que no se retire una bicicleta si la estación está vacía.
2. Que el camión reponga sólo cuando la estación está vacía.

🏙️ Ejercicio 5 – Redes de Petri

📌 Enunciado resumido (Reglas del sistema de bicicletas)

1. La estación tiene **capacidad para 5 bicicletas**.
2. Los usuarios solo pueden **retirar si hay bicicletas**.
3. El camión agrega **3 bicicletas cada vez que la estación queda vacía**.
4. El camión **no puede reponer si aún hay bicicletas**.
5. Las acciones son **atómicas**.

✏️ 5.1) Red de Petri que modela el sistema

✔ Lugares

Lugar	Significado	Marca inicial
`Bicis`	Cantidad de bicicletas en la estación	●●●●● (5 tokens)
`Espacio`	Espacios libres en la estación	0 tokens
`CamiónListo`	Camión disponible para reponer	● (1 token)

> *Solo modelamos la estación y la disponibilidad del camión. Los usuarios no se modelan como tokens, ya que solo disparan transiciones.*

✔ Transiciones

Transición	Acción
`Retirar`	Un usuario toma 1 bicicleta
`Reponer`	Camión repone 3 bicicletas

✔ Arcos y Reglas

- `Retirar`:
  - Consume: 1 token de `Bicis`.
  - Produce: (ningún token adicional).
- `Reponer`:
  - Consume: 1 token de `CamiónListo`.
  - Produce: 3 tokens en `Bicis`.
  - **Tiene inhibidor** sobre `Bicis`: solo dispara si `Bicis = 0`.

📌 **Capacidad máxima = 5**: el lugar `Bicis` tiene **capacidad máxima de 5**, por lo que el camión nunca puede agregar si supera ese valor.

📍 Diagrama

csharp Copy code

```
        (Bicis)•••••
           |  ^
           |  |
     [Retirar] |
           |   \
           v    X (inhibidor)
      (CamiónListo)• ——→ [Reponer] ——→ (Bicis)
                          (+3)
```

Leyenda

- Arco con "X" (círculo chico) = inhibidor: solo dispara si el lugar tiene **0 tokens**.
- `[Reponer]` agrega 3 tokens a `Bicis`.

🧠 5.2) Garantías del modelo

✔ 1) No se puede retirar una bicicleta si la estación está vacía

- La transición `Retirar` consume 1 token de `Bicis`.
- Si `Bicis = 0`, la transición **no puede dispararse** → no hay retirada posible.
- Esto cumple con la regla 2 del enunciado.

📌 **Explicación formal:**

$$\textit{Retirar} \text{ habilitada} \iff B > 0$$

✔ 2) El camión solo repone cuando la estación está vacía

- La transición `Reponer` tiene **arco inhibidor desde** `Bicis`.
- Por lo tanto, `Reponer` solo puede disparar si `Bicis = 0`.
- Cuando se dispara, agrega exactamente 3 bicicletas.

📌 **Explicación formal:**

$$\textit{Reponer} \text{ habilitada} \iff \textit{Bicis} = 0$$

📌 Conclusión

La red de Petri modelada:

- respeta la capacidad,
- evita retiros sin bicicletas,
- evita reposición si no está vacío,
- asegura atomicidad mediante transiciones.

3era Fecha

1) semáforo, sus funciones y variables internas
2) funciones de entrada y salida de una red de petri, y diagrama del cliente servidor
3) exclusión mutua en ambiente distribuido, diagrama y problemas
4) regla, acción, comportamiento y conocimiento de entidades
5) problema de reserva de restaurante, analizar problemas cuando está congestionado, proponer solución (locks, transacciones, etc) y proponer solución para ciertos casos particulares

Tenes reservas para restaurantes, los clientes pueden elegir
1. restaurante y fecha
2. horario

3. numero de comensales

El reservado de la mesa en si es "automatico" y se confirma en a lo sumo 5 mins, sino hay timeout. El principio me parecia se referia a que el cliente confirme pero dsps menciona timeout de pago, igual no afecto a mi respuesta ja.

5.1. Que problemas de concurrencia detectas? Detallar
5.2. Como resolverias esos problemas, detallar metodo de sincronizacion (por ejemplo con semaforos, actores, transacciones, etc).
5.3. Explicar 3 casos de como lo manejaria tu diseño
Ejemplo:
Caso 1: dos usuarios reservan la ultima mesa
Caso 2: hay un timeout, expira proceso de pago decia.
Caso 3: falla el servidor de lock de mesas.

PREGUNTA TIPO FINAL:
¿Qué es la sección crítica y qué condiciones debe cumplir una solución al problema de la sección crítica?

⇒ Una **sección crítica** es la parte del programa donde un proceso accede a recursos compartidos, y por lo tanto **no puede ejecutarse simultáneamente con otros procesos** que accedan a los mismos recursos.
Cada proceso se ejecuta en un **loop infinito**, cuyo código puede dividirse en **parte crítica** y **parte no crítica**.
Para que una solución al problema de la sección crítica sea correcta, debe cumplir estas especificaciones:

1. **Exclusión mutua:** No deben intercalarse instrucciones de la sección crítica entre procesos. Si un proceso está en la sección crítica, ningún otro puede entrar.

2. **Ausencia de deadlock:** Si dos procesos intentan entrar a la sección crítica al mismo tiempo, **eventualmente alguno debe entrar**. No puede ocurrir un bloqueo eterno.
3. **Ausencia de starvation:** Si un proceso quiere entrar a la sección crítica, **eventualmente debe tener éxito**, es decir, no puede quedar postergado indefinidamente mientras otros sí entran.

Además, se cumple que:

- La **sección crítica debe progresar** (eventualmente finalizar).
- La **sección no crítica no requiere progreso** (un proceso puede quedarse en ella o incluso bloquearse sin afectar a otros).

EJERCICIO REDES DE PETRI:

Modelar una red que:

- Los técnicos realizan diagnósticos en una base de datos de una empresa.
- La base de datos de la empresa puede tener hasta 3 técnicos a la vez realizando diagnósticos concurrentemente.
- la base de datos luego de 3 diagnósticos, debe pasar por un ingeniero para que “aplique un parche” antes de que algún técnico quiera realizar otro diagnóstico.
- Las operaciones de “iniciando diagnóstico”, “realizando diagnóstico”, “finalizando diagnóstico”, “aplicación del parche”, etc. deben ser atómicas.
- Si un ingeniero está limpiando la base de datos, no puede haber un analista realizando un análisis, y viceversa.

Describir en detalle los places, transitions, etc.

Realizar un grafo de alcance para una posible ejecucion.

Explicar work-stealing y fork join, y por que hay que usar varias colas y no una sola.

Cual es la motivacion del modelo de actores, que es el actor y los estados del actor.

como detectar deadlocks en concurrencia distribuida.

que es la entidad, su capacidad, restricciones temporales.

3 modelos de concurrencia en transacciones.

<u>Preguntas teóricas FINAL</u>

“<u>Introducción</u>”

Qué modelos de concurrencia se ven

Los modelos de concurrencia que se ven son:

- **Estado mutable/compartido**:
    - Los procesos/hilos **comparten variables** (estado global).
    - El acceso concurrente puede generar **condiciones de carrera.**
    - Requiere mecanismos explícitos de **sincronización.**

Es el modelo “clásico” de concurrencia y el más propenso a errores si no se protege correctamente el estado.

- **Paralelismo Fork-Join**: Es para un tipo específico de escenario en donde:
    - El problema se puede **subdividir en tareas independientes**.
    - Cada subtarea se ejecuta en paralelo (sin depender de la otra), siendo esta la parte del fork.
    - Al final se **combinan los resultados** para llegar al resultado final**,** join.
    - Estos procesos se suelen llamar “workers”.
    - No hay estado compartido entre workers.

  Una ventaja clave, es que no hay variables compartidas ⇒ evita **race condition.**

- **Canales/Mensajes**: Es un medio de comunicación en donde:
    - Los procesos se comunican entre ellos **enviando mensajes por canales.**
    - Los canales pueden ser **unidireccionales** o **bidireccionales**.
    - La comunicación entre procesos puede ser **sincrónica** o **asincrónica**, es decir:
        - **sincrónica**: un proceso **no** continúa hasta no terminar de comunicarse con el otro proceso.
        - **asincrónica**: un proceso continua sin importar si se completa la comunicación con otro proceso.
    - Ejemplo típico: **sockets.**

  Idea central: no se comparte memoria, se **comunica información.**
  Una abstracción más de este modelo es **Actores.**

- **Programación Asincrónica**: Modelo de **concurrencia colaborativa.**
    - El programa se divide en **tareas livianas.**
    - Un **executer** decide qué tarea se ejecuta.
    - Las tareas ceden el control a la llegada de una importante, cómo puede ser, I/O.
    - Es decir, es un paradigma en el que las operaciones no bloqueantes permiten que un programa realice otras tareas sin tener que esperar que las operaciones de I/O se completen. Sin tener que esperar pasivamente a que una operación finalice, puede seguir ejecutando otras tareas que no dependan de esa operación de I/O en particular.

- **Actores**: Es una abstracción del pasaje de mensajes.
    - Cada actor:
        - tiene su **estado privado.**
        - **NO comparten variables.**
        - tienen internamente sus variables que solo él puede modificar.
        - procesa mensajes de su casilla de correo.
        - Es como la POO que los objetos se pasan información a través de métodos.

  Propiedad fundamental: **No** existe estado mutable compartido.

De qué formas se puede proteger el estado compartido

Serializar el acceso al estado compartido:

- Los procesos se van a ejecutar al mismo tiempo pero, habrá un **conjunto de procedimientos** que requieren ejecutarse de modo que sólo la ejecución de un procedimiento puede hacerse a la vez.
- Si existe algún procedimiento en el conjunto ejecutándose y **otro** quiere ejecutarse a la vez, este **último** deberá esperar hasta que el primero termine de ejecutarse.
- Serializar para controlar el acceso a las variables compartidas (puede pensarse como variables globales).
- Podemos marcar regiones de código que no pueden superponerse en la ejecución al mismo tiempo (sección crítica). El resto del código puede ejecutarse concurrentemente.

"<u>Fork-Join</u>"

Qué es fork join

Es un modelo de concurrencia que consiste en dividir tareas en subtareas que no dependan unas de otras, se ejecutan de forma paralela siendo esta la seccion "fork" y al finalizar todas, se combinan todos los resultados, siendo esta la seccion "join" para completar la ejecución, evitando el uso de estado compartido y sin race conditions.

Cómo se implementa fork join

Fork-Join se implementa mediante un conjunto de threads "workers", donde cada uno mantiene su propia cola de tareas. Las tareas se subdividen recursivamente y se encolan para su ejecución. Los workers toman tareas de su cola y, si se quedan sin trabajo, utilizan un mecanismo de "work stealing" para robar tareas de otros threads. Al finalizar las subtareas, se sincronizan en el join para combinar los resultados. En Rust puede implementarse usando librerías como Rayon.
No necesariamente todas las implementaciones tienen work stealing, pero hoy en día la mayoría lo tiene dado su eficiencia frente a tener una cola única de tareas.

Cuál es la performance ideal de fork join

La performance ideal de Fork-Join es un speedup lineal, donde el tiempo de ejecución es aproximadamente el tiempo secuencial dividido por la cantidad de threads, es

decir $\mathbf{t_{secuencial}}$ **/** $\mathbf{N_{threads}}$**,** siempre que las tareas estén bien balanceadas y el costo de sincronización y combinación de resultados sea despreciable.
¿Qué quiere decir esto? Tiempo secuencial: tiempo total de ejecución con un solo thread el total de las tareas. Esto se divide por la cantidad de threads a utilizar.

Qué desventajas tiene fork join

Fork-Join presenta desventajas cuando las tareas no están bien balanceadas, ya que algunas pueden tardar más que otras, y cuando el costo de sincronización y de combinación de los resultados en el join es significativo (tener que esperar a que todas las tareas finalicen). Además, este modelo no es aplicable a todos los problemas (el problema debe si o si poder subdividirse en tareas independientes).

Cómo funciona work stealing

Cada thread va a tener su propia cola de dos extremos de tareas en donde, cada thread cuando quiere realizar una tarea ⇒ desencola del extremo final la tarea, la subdivide y encola al final, dejando las tareas más viejas en el extremo inicial de la cola. Cuando un thread se queda sin trabajos en su cola, entonces, aleatoriamente elige una cola de un thread compañero, en donde desencola una tarea desde el inicio, siendo esta la más vieja y reduciendo la probabilidad de que desencole la misma tarea que el thread original al mismo tiempo.

Ejemplo de uso Fork-Join

Por ejemplo, para procesar muchas imágenes o archivos de forma independiente, se puede usar Fork-Join, dividiendo el trabajo en partes que se ejecutan en paralelo y luego combinando los resultados.

“<u>Programación Asincrónica</u>”

Cuándo se usa programación asincrónica

La programación asincrónica se utiliza cuando hay muchas tareas livianas, especialmente con operaciones de I/O, para evitar crear muchos threads y bloquear recursos. Tiene la particularidad de que no necesita esperar la respuesta a una operación de I/O para continuar, puede seguir con alguna otra tarea que no dependa de esta respuesta hasta que, esa respuesta llegue y siga con esa tarea anterior.

Qué es un future

Un future es una estructura que representa el resultado de una computación asincrónica que **todavía no terminó,** pero que estará disponible más adelante.

Qué es una expresión wait

await suspende la ejecución de una tarea hasta que un future esté listo, sin bloquear el thread. Durante ese tiempo, el control vuelve al executer, permitiendo que se ejecuten otras tareas en el thread, y luego la tarea se reanuda cuando el resultado del future está disponible.

Qué es una tarea

Una tarea es una unidad liviana de ejecución asincrónica que es administrada por un executer y se ejecuta sobre uno o más threads del sistema. Puede suspenderse y reanudarse mediante await, permitiendo que múltiples tareas compartan eficientemente los mismos recursos.

Qué son los tipos Pin

Los tipos **Pin** garantizan que un objeto no cambie de dirección en memoria, lo cual es necesario para el correcto funcionamiento de futures. Esto es importante porque muchos futures contienen referencias internas a sus propios datos, y si el objeto se mueve en memoria dichas referencias quedarían invalidas. Por ejemplo, al usar **Pin<Box<T>>**, el objeto queda fijado en una dirección estable, evitando estos problemas.
"¿Por qué Rust necesita Pin en async?"
⇒ Porque los futures generados pueden ser auto-referenciados y no deben moverse en memoria.

Qué es el modelo piñata

El modelo piñata describe como una tarea asincrónica se va dividiendo en partes que se ejecutan entre distintos **await,** permitiendo intercalar ejecuciones. En cada **await,** la tarea se "abre" y se suspende, devolviendo el control al executer, que puede ejecutar otras tareas. De esta forma, se intercalan múltiples ejecuciones sin bloquear los threads.

Por ejemplo, en un servidor web que atiende a miles de usuarios, cada pedido tiene que esperar respuestas de base de datos o de red. Con el modelo piñata, cuando una tarea llega a un **await**, se suspende y el executer puede atender a otros usuarios. Luego, cuando llega la respuesta, la tarea se reanuda. Esto permite manejar muchas conexiones con pocos threads.

Qué hace async

Async indica que una función es asincrónica y que su resultado será un future.

Qué hace await

await espera el resultado de un future sin bloquear el thread, permitiendo que se ejecuten otras tareas.

Qué hace block_on

block_on ejecuta una función asincrónica de forma sincrónica, bloqueando el thread hasta obtener el resultado, y no debe usarse dentro de funciones async.

Cuándo usar programación asíncrona

La programación asincrónica se usa cuando hay muchas tareas concurrentes que pasan gran parte del tiempo esperando operaciones de I/O, y se busca evitar el uso excesivo de threads para manejar muchas tareas sin bloquearlos. No es adecuada para tareas intensivas de cómputo, sino para aplicaciones con mucha espera y alta concurrencia.

Ejemplo claro para Programación Asincrónica

Por ejemplo, en un servidor web con muchas conexiones simultáneas, la programación asincrónica permite atender muchos usuarios usando pocos threads, ya que cuando una tarea espera una operación de I/O se suspende y deja ejecutar a otra.

Comparar el modelo de Programación Asincrónica con el modelo tradicional de threads o de procesos del Sistema Operativo. Describir ventajas y desventajas.

En el modelo tradicional, cada tarea se ejecuta en un thread o proceso administrado por el sistema operativo, y cuando realiza una operación de I/O queda bloqueada. Esto simplifica la programación, pero consume más recursos y es la peor con muchas tareas. En cambio, en el modelo asincrónico las tareas son livianas y administradas por un executer. Cuando esperan I/O se suspenden sin bloquear el thread, permitiendo manejar muchas tareas con pocos recursos, aunque con mayor complejidad de programación.

Describir un ejemplo de caso de uso en el que convenga utilizar el modelo asincrónico y otro ejemplo para el modelo de threads. Justificar cada elección.

Un ejemplo donde conviene usar programación asincrónica es un servidor web con muchas conexiones simultáneas, ya que las tareas pasan mucho tiempo esperando operaciones de I/O y pueden suspenderse sin bloquear threads. En cambio, para un procesamiento intensivo de datos o imágenes, conviene usar threads, ya que permiten aprovechar los múltiples núcleos del procesador para ejecutar cómputo paralelo que no dependen de ningún future.

“<u>Corrección</u>”

Qué propiedades de corrección existen

Las propiedades de corrección se dividen en safety, que garantizan que no ocurran estados incorrectos (Exclusión mutua y ausencia de deadlocks), y liveness, que aseguran que el sistema progrese y que las solicitudes sean eventualmente atendidas (Ausencia de starvation y fairness).

Qué se debe cumplir para que suceda un deadlock

Un deadlock ocurre cuando se cumplen simultáneamente cuatro condiciones: exclusión mutua, retención y espera, no apropiación de recursos y espera circular, lo que provoca que los procesos queden bloqueados indefinidamente esperando recursos que poseen otros.

Qué es un deadlock

Un deadlock es una situación en la que dos o más procesos a la vez, dentro de cada sección crítica, un proceso 1 necesita la obtención de un recurso que el otro proceso 2 tiene bloqueado, para continuar y viceversa. Esto sucede en simultáneo, por ende, ambos quedan esperando indefinidamente siendo esto una espera circular.

Cómo se implementan los locks en Unix

En Unix, los locks se implementan principalmente mediante bloques sobre archivos del sistema de archivos. Estos locks son de tipo advisory, es decir, no son impuestos por el sistema operativo, sino que los procesos deben respetarlos voluntariamente.

Existen dos tipos principales: locks compartidos o de lectura, que permiten que varios procesos accedan simultáneamente, y locks exclusivos o de escritura, que permiten el acceso de un solo proceso.

Para obtener un lock de lectura, el proceso debe esperar a que se liberen los locks de escritura, y para obtener un lock de escritura debe esperar a que se liberen todos los locks existentes de ambos tipos. De esta forma se controla el acceso concurrente a los recursos.

Cómo se implementan los locks en Rust

En Rust, los locks se implementan mediante estructuras del módulo **std::sync**, como **Mutex** y **RwLock**, que permiten proteger datos compartidos entre threads. El acceso está regulado por los traits **Send** y **Sync**, que garantizan que los tipos puedan transferirse y compartirse de forma segura entre threads.

Rust provee locks compartidos de lectura y locks exclusivos de escritura mediante **RwLock**, utilizando los métodos **read** y **write**. Estos métodos bloquean el thread hasta poder obtener el lock y devuelven un guard que libera automáticamente el lock mediante el mecanismo RAII al salir del scope.

Para poder compartir datos entre threads, se requiere que el tipo protegido implemente **Send** y **Sync**, garantizando seguridad en tiempo de compilación.

Qué tipos de locks tiene Rust

Rust provee principalmente **Mutex** para exclusión mutua y **RwLock** para acceso concurrente con múltiples lectores y un solo escritor. Además, existen mecanismos como **OnceLock** para inicialización segura, y otros locks en librerías externas como spinlocks. Todos utilizan guards y RAII para liberar automáticamente los recursos.

Qué es un lock envenenado

Un lock queda en estado envenenado cuando un thread lo toma de forma exclusiva (write lock) y mientras tiene tomado el lock, ejecuta panic! (dejando los datos en un posible estado inconsistente). Las llamadas siguientes de read y write sobre el mismo lock, devolverán Err para advertir a los demás threads sobre esta situación.

“<u>Sincronización</u>”

Cuáles son los estados de ejecución de un proceso

Un proceso puede estar en 5 estados distintos: nuevo, listo, en ejecución, en espera o bloqueado, y terminado. El scheduler se encarga de moverlo entre estos estados según la disponibilidad de CPU y los eventos del sistema.
Un proceso puede finalizar por decisión propia o por otro proceso.

Cuando un proceso termina mediante **exit**, no desaparece inmediatamente, sino que queda en un estado “zombie”, en el cual ya liberó su espacio reservado y demás, cuando otro proceso pregunta por él sigue respondiendo hasta que pregunta su proceso padre, en este caso muere por completo.

Qué es un semáforo y qué operaciones tiene

Un semáforo es un mecanismo de sincronización implementado como un tipo de dato compuesto por dos campos: un contador entero no negativo y un conjunto de procesos bloqueados.

El contador indica la cantidad de recursos disponibles, mientras que el conjunto almacena los procesos que están esperando. Sus operaciones **wait** y **signal** modifican estos campos de forma atómica, bloqueando o despertando procesos según corresponda. El sistema operativo nos asegura que las operaciones **wait** y **signal** son **atómicas**.

La operación **wait** resta 1 al contador, obteniendo acceso al recurso en caso de que el contador sea > 0, si es = 0 se bloquea en espera.
La operación **signal** suma 1 al contador, devolviendo un acceso al recurso.

Una vez que un proceso realiza **signal** sobre el semáforo, se le otorga el acceso de manera al azar a algún proceso que haya realizado **wait** y no por orden.

Qué es un mutex y qué operaciones tiene

Un mutex es un mecanismo de sincronización que garantiza la exclusión mutua en el acceso a una sección crítica, permitiendo que solo un proceso o thread pueda acceder al recurso compartido a la vez. A diferencia del semáforo, el mutex tiene noción del dueño: únicamente el proceso que lo adquirió puede liberarlo.

Sus operaciones principales son **lock** y **unlock**.

La operación **lock** intenta tomar el mutex; si este se encuentra libre, el proceso lo adquiere y continua su ejecución, y si está ocupado, el proceso queda bloqueado hasta que sea liberado.

La operación **unlock** libera el mutex, permitiendo que otro proceso bloqueado pueda adquirirlo. El sistema operativo garantiza que estas operaciones sean atómicas.

Depende de la implementación pero en Linux se utilizan colas FIFO o prioridades para los procesos que quedan esperando el lock.

Cómo es el problema del productor consumidor

Es un problema de sincronización en el que uno o más procesos productores generan elementos y lo depositan en un buffer compartido, el cual puede ser acotado, mientras que uno o más procesos consumidores retiran esos elementos para procesarlos.

El productor, al ser un buffer acotado, no puede producir si este está lleno y el consumidor no podrá consumir si está vacío. Además, se debe garantizar que el acceso al buffer sea mutuamente excluyente para evitar race condition.

Cómo se resuelve el problema del productor consumidor con semáforos

Se resuelve con dos semáforos: un semáforo que indique el consumo y otro semáforo que indique la carga de producto. El consumidor realiza un wait al semáforo “notEmpty” para consumir, consume y realiza signal al semáforo “notFull”. El productor realiza wait al semáforo “notFull”, deposita el producto y realiza signal a “notEmpty”.

Qué es una barrera y qué operaciones tiene

Una barrera es un mecanismo de sincronización que permite coordinar un conjunto de procesos o threads, obligándolos a esperar en un punto determinado del programa hasta que todos hayan llegado a ese punto.

Cada proceso que alcanza la barrera ejecuta una operación de espera sobre ella. Si todavía no llegaron todos los procesos, queda bloqueado. Cuando el último proceso alcanza la barrera, se liberan todos los procesos bloqueados y pueden continuar su ejecución.

La barrera posee esencialmente una única operación de espera.

Qué es un monitor y qué operaciones tiene

Un monitor es una herramienta de sincronización que garantiza exclusión mutua y permite que los procesos se bloqueen hasta que se cumpla una condición. Solo un proceso puede ejecutar dentro del monitor a la vez.

El monitor utiliza variables de condición, que no almacenan valores sino que mantienen una cola FIFO de procesos bloqueados.

Estas variables poseen tres operaciones atómicas: **waitC**, que bloquea el proceso y libera el monitor, **signalC**, que despierta a un proceso bloqueado, y **empty**, que indica si hay procesos esperando.

Por ejemplo, una sala con cupos limitados. Los alumnos entran a un monitor, si está llena se bloquean con waitC, y cuando alguien se va se hace signalC para que entre otro.

Qué compone a un monitor

Un monitor está compuesto por un nombre, un conjunto de variables internas, procedimientos del monitor que acceden directamente a esas variables, una interfaz pública para que los procesos puedan acceder, la inicialización de las variables internas y un conjunto de variables de condición que incorporan sincronización al monitor.

Qué es un condition variable y qué operaciones tiene

Una condition variable es un mecanismo de sincronización que se utiliza dentro de un monitor para permitir que los procesos se bloqueen hasta que se cumpla una determinada condición.

No almacena ningún valor propio, sino que mantiene una cola FIFO de procesos que están esperando. Su función es coordinar la ejecución entre procesos mediante espera y señalización.

Posee tres operaciones atómicas: **waitC**, **signalC** y **empty**.

una condition variable es lo mismo que un monitor?

No, una condition variable no es lo mismo que un monitor. El monitor es una estructura que encapsula datos, procedimientos y exclusión mutua, mientras que las condition variable es un mecanismo que forma parte del monitor y permite que los procesos se bloqueen y sean despetados cuando se cumple una condición.

Monitor ⇒ sala de control.
Condition ⇒ lista de espera.

Enumerar las similitudes y diferencias que se pueden encontrar entre un semáforo y un monitor.

Tanto los semáforos como los monitores son mecanismos de sincronización que permiten coordinar procesos concurrentes y controlar el acceso a recursos compartidos, evitando race conditions.

Sin embargo, presentan diferencias importantes en el comportamiento de sus operaciones.

En los semáforos, la operación **wait** puede o no bloquear dependiendo del valor del contador, mientras que en los monitores la operación **waitC** siempre bloquea el proceso.

En los semáforos, la operación **signal** siempre tiene efecto, ya sea incrementando el contador o despertando a un proceso, mientras que en los monitores **signalC** no tiene efecto si la cola está vacía.

Además, en los semáforos, **signal** desbloquea a un proceso arbitrario, mientras que en los monitores **signalC** desbloquea al proceso que se encuentra al comienzo de la cola FIFO.

Por último, en los semáforos, un proceso desbloqueado mediante **signal** puede continuar su ejecución inmediatamente, mientras que en los monitores, un proceso despertado con **signalC** debe esperar a que el proceso señalizador abandone el monitor antes de continuar.

Describir un ejemplo de situación en el que sea conveniente utilizar un monitor en lugar de un semáforo. Justificar la elección.

Un ejemplo en el que conviene utilizar un monitor en lugar de un semáforo es en un sistema que administra un recurso compartido complejo, como una cola de impresión o un buffer con múltiples condiciones de acceso.
Por ejemplo, en una cola de impresión, varios procesos envían trabajos para imprimir y otros los procesan. Es necesario controlar el acceso a la cola, permitir que los procesos esperen cuando está vacía y despertarlos cuando hay nuevos trabajos.

En este caso, un monitor permite encapsular la estructura de la cola y sus operaciones dentro de una única estructura, garantizando exclusión mutua automática y utilizando condition variables para sincronizar la espera y la señalización.

Esto reduce la posibilidad de errores, ya que el programador no debe manejar explícitamente las operaciones **wait** y **signal** como en los semáforos. Además, mejora la claridad y el mantenimiento del código.

Por estas razones, en situaciones donde el recurso compartido tiene múltiples operaciones y condiciones asociadas, es preferible utilizar monitores en lugar de semáforos.

Describir un ejemplo de situación en el que sea conveniente utilizar semáforos en lugar de un monitor. Justificar la elección.

Un ejemplo en el que conviene utilizar semáforos en lugar de monitores es en el control de acceso a un conjunto limitado de recursos idénticos, como por ejemplo una cantidad fija de impresoras, conexiones a una base de datos o lugares en un estacionamiento.

En este caso, varios procesos necesitan acceder a un recurso, pero solo puede haber un número limitado de accesos simultáneos. Un semáforo contador permite representar directamente la cantidad de recursos disponibles mediante su contador.

Cada proceso ejecuta **wait** antes de utilizar un recurso, decrementando el contador y ejecuta **signal** al liberarlo, incrementando. Si no hay recursos disponibles, el proceso queda bloqueado automáticamente.

En este tipo de situaciones, el uso de semáforos resulta más simple y natural que un monitor, ya que no se requiere encapsular estructuras complejas ni manejar múltiples condiciones, sino únicamente controlar la cantidad de accesos concurrentes.

Por estas razones, cuando se trata de administrar un número fijo de recursos idénticos, es conveniente utilizar semáforos en lugar de monitores.

Diferencia top entre Semáforos y Monitores

En el caso de los semáforos, una vez que el proceso obtiene el permiso mediante **wait**, el mecanismo no controla cómo utilizar los datos, quedando a criterio del proceso. En

cambio, en los monitores, el acceso a los datos está encapsulado en procedimientos, lo que impone una forma controlada de utilización.

“Redes de Petri”

Qué es una red de Petri

Es un modelo matemático y gráfico para representar sistemas concurrentes, donde hay: procesos en paralelo, sincronización, recursos compartidos y bloqueos. Se usa mucho para analizar deadlocks, exclusión mutua, sincronización y concurrencia.
Se compone por Lugares(círculos): estados, recursos o condiciones; Transacciones(rectángulos): acciones o eventos; y Tokens(marcas): el estado actual del sistema.

No todos los problemas pueden modelarse de forma práctica con redes de Petri básicas. Si bien existen extensiones que permiten representar sistemas complejos, las redes clásicas se utilizan principalmente para modelar concurrencia, sincronización y recursos compartidos. Además, en muchos casos el modelo se vuelve demasiado grande o genera explosion de estados.

Qué problemas clásicos de la materia se pueden modelar con redes de Petri

Se pueden modelar problemas clásicos como productor-consumidor (con buffer infinito o acotado), cliente-servidor, exclusión mutua, fork-join, detección de deadlocks y sincronización de procesos, permitiendo analizar sus estados y posibles bloqueos.

“Actores”

Qué es el modelo de actores

El modelo de actores es un modelo de concurrencia en el cual el sistema está compuesto por entidades llamadas actores, que se comunican exclusivamente mediante el envío de mensajes. Cada actor procesa los mensajes de forma secuencial y no comparte memoria con otros actores.

En este modelo: No hay estado compartido, no hay locks y la sincronización se logra a través de mensajes.

Cómo puede ser la comunicación entre actores

La comunicación entre actores se realiza únicamente mediante el envío de mensajes, de forma asincrónica (tienen un buffer), a través de canales o direcciones asociadas a cada actor.

Qué posee cada actor

El actor es la primitiva principal del modelo. Son livianos y se pueden crear miles.

Cada actor posee un estado interno privado (NO SE COMPARTE), un conjunto de comportamientos que son los mensajes o handlers que tiene el actor para contestar los mensajes que recibe. El estado son las variables internas que define, no se comparten. Cada actor cuenta con una dirección en donde va a recibir los mensajes y una casilla de mensajes que es una FIFO en donde tendrá los últimos mensajes recibidos.

Qué es un mensaje

Un mensaje es una unidad de información que un actor envía a otro actor para solicitar una acción, transferir datos o notificar un evento. Son estructuras simples inmutables, con esto nos aseguramos que no haya race condición por mutar el mismo elemento sino que, la memoria no se va a modificar.

Qué posee un canal

Un canal posee dos extremos: un emisor y un receptor. Una parte del código envía mensajes a través del extremo transmisor, y otra parte consulta del extremo receptor para recibirlos.

Los canales permiten múltiples productores y un único consumidor, transfieren el ownership del elemento enviado y permiten clonar el extremo de envío para crear varios emisores.

Qué no comparte un actor

Un actor no comparte estado interno ni su memoria con otros actores. No comparte sus variables internas, como se implementan sus handlers que son el comportamiento ni tampoco su buffer en donde recibe sus mensajes. Cada actor accede únicamente a su propio estado y se comunica exclusivamente mediante el envío de mensajes.

Esto implica que: no hay race condition, no hay locks y no hay secciones críticas.

De qué formas se puede enviar un mensaje

Hay varias formas de enviar un mensaje:

- **do_send(M):** Ignora errores en el envío del mensaje. Si la casilla de mensajes está cerrada, se descarta. No retorna el resultado.
- **try_send(M):** Trata de enviar el mensaje inmediatamente. Si la casilla de mensajes está cerrada o llena, retorna error.
- **send(M):** Envía el mensaje y retorna un objeto **FUture** con el resultado del procedimiento.

Qué es el contexto

Los actores mantienen el contexto interno de ejecución, o estado. Permite al actor determinar su dirección, cambiar los límites de la casilla de mensajes o detenerse. Los mensajes llegan a la casilla primero, luego el contexto de ejecución llama al handler específico.

Qué es el arbiter

Provee el contexto de ejecución asincronico para los actores, funciones y futures. El arbiter asigna recursos y decide en qué thread o contexto se ejecuta cada uno. Su función es crear actores, planificar ejecución, asignar threads y administrar el runtime.

"Exclusión Mutua Distribuida"

Qué algoritmos de exclusión mutua distribuida existen

Existen 3 ⇒ Algoritmo centralizado, distribuido y token ring.

Algoritmo centralizado: Un ejemplo práctico es una impresora compartida en una oficina. Un servidor central administra la cola de impresión y otorga permisos para imprimir, evitando que varios equipos usen la impresora al mismo tiempo.

Algoritmo distribuido: Un ejemplo práctico es una red de sensores sin servidor central que comparten una antena de transmisión. Cada sensor solicita permiso a los demás antes de transmitir, y se coordina mediante mensajes y timestamps.

Algoritmo Token Ring: Un ejemplo práctico puede ser un sistema de control en una fábrica automatizada, es una línea de producción donde varias máquinas deben modificar un parámetro compartido. El token circula entre ellas y solo la que lo posee puede realizar cambios.

Qué es un algoritmo centralizado

El algoritmo se basa en que existe un proceso designado como "líder", el cual se encarga de controlar la sección crítica. Si un proceso de una computadora quiere entrar a su sección crítica, primero envía una solicitud con el líder para saber si puede entrar o no. Si no hay nadie en la sección crítica, el líder contesta OK, y sino, espera hasta que se libere para responder la petición con OK.
Puede pasar que el líder caiga y se tenga que realizar una reelección de quien será líder, para eso hay algoritmos que se ven después…

Conviene para sistemas chicos, servidor confiable, infraestructura central, etc.

Qué es un algoritmo distribuido

A diferencia del centralizado, acá no hay un líder y todos siguen teniendo los mismos derechos de querer entrar a la sección crítica, entonces, cuando un proceso quiere entrar a la sección crítica deberá enviar una petición a cada proceso que integre el sistema distribuido un mensaje de petición para poder entrar, ya que no se sabe quién está en la sección crítica. Cada proceso cuando recibe una petición, responde con OK si no está en la sección crítica, responde luego si está en la sección crítica, y si también quisiera entrar, es decir, hay más de un proceso que quiere entrar a la sección crítica ⇒ se comparan los timestamps de las peticiones y claramente el menor gana y las demás se postergan.

Conviene para Redes sin servidor, sistemas P2P, sensores, etc.

Qué es un algoritmo de token ring

En este algoritmo un proceso cuando quiere entrar a la sección crítica, no tiene que preguntarle a ningún proceso si puede entrar ni tampoco consultarle a un “líder”, simplemente esperar. Si esperar, porque todos los procesos que integren el sistema distribuido van a estar conectados en forma de “anillo” y se va a pasar en cadena un “token” el cual te da la posibilidad de entrar a la sección crítica. Cuando un proceso recibe el token se fija, si necesita entrar a la sección crítica lo usa y sino lo sigue pasando. De esta manera no sabemos quién está en la sección crítica pero tampoco nos interesa, simplemente esperamos a recibir el token. Este token no se permite guardar para “cuando necesite” y en caso de que un proceso falle cuando tiene el token, se deberá detectar y generar otro, en caso contrario cae todo el sistema.
Para detectar la falla se implementa un sistema de detección de fallas, como ping/pong entre procesos, definir una cota máxima de timeout a una vuelta completa, etc.

Conviene para Redes estables, nodos fijos, sistemas industriales, etc.

“<u>Modelo Cliente-Servidor</u>”

Qué tipos de arquitectura existen

En el modelo Cliente-Servidor vimos dos arquitecturas:

- **Arquitectura de dos niveles:** El cliente y el servidor tienen contacto directo.
- **Arquitectura de 3 niveles:** Middleware
    - Capa de software ubicada entre el cliente y el servidor.
    - Provee principalmente seguridad y balanceo de cargas.

Qué tipos de servidor existen

En el modelo Cliente-Servidor vimos dos tipos de servidores:

- **Iterativo:** en donde responde peticiones una a la vez.
- **Concurrente:** puede responder más de una petición a la vez.

Qué capas componen el modelo OSI

El modelo OSI está compuesto por 7 capas:

- **Física:** Se encarga de la transmisión de datos a través del medio físico.
- **Enlace:** Se encarga de la transmisión de los datos libre de errores entre dos nodos conectados físicamente. Organiza los bits en frames y maneja la detección de errores.
- **Red:** Se encarga de definir la ruta que van a seguir los datos hasta el destino. Gestiona el direccionamiento lógico y enrutamiento de paquetes entre redes.
- **Transporte:** Garantiza la comunicación confiable entre dispositivos (si es TCP). Divide el paquete en fragmentos de ser necesario, control de flujo, etc.
- **Sesión:** Administra el diálogo entre aplicaciones. Establece, mantiene y finaliza sesiones de comunicación, sincronización, etc.
- **Presentación:** Traduce, cifra o comprime los datos para que sean comprensibles entre sistemas diferentes.
- **Aplicación:** Proporciona servicios de red directamente a las aplicaciones del usuario. Es la interfaz entre el software y la red.

“<u>Algoritmos de Elección</u>”

Qué algoritmos de elección existen

Varios algoritmos requieren de un líder y en este caso, se necesita un criterio para la elección en primera instancia o cuando se necesita una re-elección. En la mayoría de casos no nos importa que proceso es el líder sino que se cubra el puesto, pero para los algoritmos asumimos que cada proceso tiene un ID único que lo identifica y que los demás procesos tienen conocimiento del mismo.
Los algoritmos que vimos en la materia son el algoritmo de Bully y el algoritmo Ring.

Un ejemplo práctico para el algoritmo Bully es un cluster de servidores con prioridades donde el nodo más potente debe ser el coordinador. El algoritmo Bully garantiza que el proceso con mayor identificador sea elegido como líder. Garantiza que se releve rápidamente el nuevo líder, ya que todos los nodos saben si tienen un nodo con mayor ID, la desventaja es la cantidad de mensajes que se mandan.

Un ejemplo práctico para el algoritmo Ring es una red de dispositivos en forma de anillo, como sensores industriales, donde el algoritmo Ring permite elegir un coordinador con bajo tráfico de mensajes. Conviene porque la red ya está organizada en anillo, son pocos mensajes y tiene un bajo consumo para algo que son “sensores” y no computadoras de mejor funcionamiento.

Bully conviene cuando hay jerarquía entre nodos y quiero que siempre gane el de mayor capacidad rápidamente, aunque genere más tráfico. Ring conviene cuando la red es estable y chica, porque usa pocos mensajes, aunque sea más lento y dependiente del anillo.

Qué es un algoritmo Bully

Asumimos que cada proceso tiene un ID único y que los demás procesos tienen conocimiento del mismo. Un proceso A se da cuenta que cayó el líder, entonces manda un mensaje de que quiere convertirse en líder pero, se lo manda solamente a los procesos con ID mayor a él, si alguno le responde OK, significa que ese otro quiere convertirse en líder y le gana por tener ID mayor. En caso de que ninguno le conteste, se autoproclama líder y manda un mensaje a cada proceso de que es el nuevo líder.
Es posible que esta elección la haga más de un proceso a la vez.

Conviene este algoritmo cuando: Tengo servidores con distinta potencia; Quiero que gane el mejor; Red confiable; No me molesta el tráfico.
Ej: data centers, cloud, clusters.

Qué es un algoritmo Ring

Asumimos que cada proceso tiene un ID único y que los demás procesos tienen conocimiento del mismo. En este caso todos los procesos estarán conectados en forma de “anillo”, cada proceso sabe cual es su anterior y cuál es su siguiente. Cuando un proceso A se da cuenta que el líder cayó, arma un ticket en donde pone su ID (ELECTION(ID)) y lo pasa al siguiente. Cada proceso pone su ID en el ticket, cuando el proceso A recibe el ticket con IDs, se fija si esta el suyo y si esta, se fija quién es el líder y manda un mensaje de COORDINATOR(maxID).

Si más de un proceso se da cuenta la caída y realiza otro ticket más, no pasa nada, eventualmente convergen.

Conviene cuando: Tengo red chica; Topología fija; Bajo consumo; Simplicidad.
Ej: sensores, laboratorios.

"Transacciones"

Qué es una transacción

Una transacción es una secuencia de operaciones que se ejecutan como una sola unidad lógica de trabajo. O se ejecuta completamente o no se ejecuta en lo absoluto, no puede quedar a medias.

Se usa principalmente en sistemas distribuidos y bases de datos para garantizar la consistencia.

Qué primitivas tienen las transacciones

Tiene las siguientes primitivas, cada una es atómica y se realiza sobre la memoria:
BEGIN: Para iniciar la transacción.
END: Para finalizar la transacción y tratar de hacer commit.
ABORT: Falla en la transacción y se deben volver los valores anteriores.
READ: Lee del archivo.
WRITE: Escribe en el archivo.

Qué son las propiedades ACID

Las propiedades ACID garantizan el correcto funcionamiento:
Atomicidad: Se ejecuta todo o nada.
Consistencia: Deja al sistema en un estado válido.
Aislamiento: No interfiere con otras transacciones.
Durabilidad: Una vez commiteados los cambios, son permanentes.

Qué algoritmos de implementación de transacciones existen

Los algoritmos que vemos en la materia de implementación de transacciones son: Private Workspace, Write Ahead Log, Commit en dos fases, Two-phase locking, Concurrencia Optimista y Timestamps.

Qué es un private workspace

Se basa en que al iniciar la transacción, el proceso recibe una copia de todos los archivos a los que tiene acceso, realiza todas sus modificaciones pero trabajando en la copia, al hacer commit, se persisten los datos. La desventaja de esto es que ocupa mucha memoria, pensar que hay que copiar todos los archivos. Si aborta, simplemente se descarta.

Qué es un write ahead log

Se realiza la transacción de manera inplace pero sumado a un log en el que, se anota cada operación, ya sea iniciar con BEGIN, COMMIT, WRITE, etc. Hasta ABORT por si

en caso de abortar, lo único que se tiene que hacer es rehacer los cambios con el log leyéndolo del final hacia el inicio. Primero se guarda en el log ⇒ despues en el storage.

Qué es un algoritmo de commit en dos fases

El coordinador es aquel proceso que realiza la transacción.

Fase 1: El coordinador escribe prepare en su log y envía el mensaje prepare al resto de los procesos. Los procesos que reciben el mensaje, escriben ready en el log y envían ready al coordinador.

Fase 2: El coordinador realiza los cambios, envía el mensaje commit al resto de procesos. Los procesos que reciben el mensaje escriben commit en el log y envían ready al coordinador.

Qué es un algoritmo de two phase locking

Cada proceso que quiere realizar una transacción se va a dividir en dos fases:
La obtención de locks: Crecimiento, solo te toma locks y no se libera (se pueden realizar operaciones entre la obtención de locks).
La liberación de locks: Decrecimiento, una vez liberado un lock, solo puede liberar y no puede obtener ningún lock hasta realizar commit o abort.
Garantiza serialización pero, puede producir deadlocks, por ejemplo, Proceso 1 quiere locks A y luego B, Proceso 2 quiere locks B y luego A ⇒ espera infinita.

Qué es un algoritmo de optimistic concurrency control

El proceso modifica los archivos sin pedir locks sobre los mismos, esperando que no haya conflictos. Al commitear, verifica si las demás transacciones modificaron los mismos archivos, si es así aborta la transacción.

¿Cuál es la diferencia con Private Workspace? Ambos usan una copia privada pero, Private Workspace no valida si las demás transacciones modificaron el mismo archivo, commitea y listo!

Qué es un algoritmo de timestamps

Cada archivo va a tener dos timestamps: de lectura y escritura.
Un proceso al iniciar una transacción recibe un timestamp global.
Cuando quiere acceder a un archivo, compara su timestamp con el del archivo:

- Si el timestamp del archivo es menor que el del proceso ⇒ Realiza la operación correctamente.
- En caso contrario ⇒ aborta la transacción.

Con esto nos evitamos el uso de locks.

Qué algoritmos de detección de deadlocks distribuidos existen

En ambientes distribuidos la detección de deadlocks es más complicada, ya que dependen de la Red, y por este problema pueden existir falsos deadlocks. En la materia vimos dos algoritmos que son el **centralizado** y el **distribuido.**

Qué es un algoritmo centralizado

En este algoritmo centralizado ya contamos con un líder/coordinador que recibe información de los procesos sobre los recursos que poseen y los que están esperando, por el cual, va a ser el encargado de saber si existe un deadlock o no. El líder se va a ir generando un grafo con nodos como procesos, los cuales van a estar conectados con otros nodos si es que esperan un lock que tienen ellos. Si el grafo llega a ser cíclico ⇒ existe deadlock.
Como ventaja es simple y como desventaja es un único punto de falla y puede ser cuello de botella.

Qué es un algoritmo distribuido

En este caso no hay un líder, por lo que, cada proceso va a ir armando su grafo a medida que recibe peticiones de recursos que poseen. Si un proceso recibe una petición de un proceso al cual, le realice una petición por un recurso, hay ciclo ⇒ es un deadlock.

PREVENCIÓN

Qué es un algoritmo de wait die

Cada transacción al iniciarse va a tener un timestamp global. Cuando un Proceso A quiera un recurso que está bloqueado por otro Proceso B, compara su timestamp con el de ese proceso:

- Timestamp Proceso A < Proceso B ⇒ Espera a que termine el Proceso B (Proceso A es más viejo que el Proceso B)
- Timestamp Proceso A > Proceso B ⇒ Se aborta la transacción del Proceso A porque es más nueva que la del Proceso B y por ende, es menos trabajo que abortar la que realizo más modificaciones.

Qué es un algoritmo de wound wait

Cada transacción al iniciarse va a tener un timestamp global. Cuando un Proceso A quiere un recurso que está bloqueado por otro Proceso B, compara su timestamp con el de ese proceso:

- Timestamp Proceso A < Proceso B ⇒ El Proceso A aborta la transacción del Proceso B (la transacción del Proceso A es más vieja que la del Proceso B, menor trabajo de deshacer).
- Timestamp Proceso A > Proceso B ⇒ Espera a que termine.

“<u>Ambientes Distribuidos</u>”

Qué es una entidad

Es la unidad de cómputo de un sistema distribuido.
Puede ser un proceso, un hilo, un actor, una máquina o cualquier componente que participa del sistema y ejecuta acciones.

Qué capacidades tiene una entidad

Una entidad puede:

- Mantener un estado interno.

- Procesar información.
- Recibir eventos externos.
- Ejecutar acciones.
- Comunicarse con otras entidades.

Estas capacidades le permiten participar en el sistema distribuido.

Qué eventos externos puede recibir una entidad

La entidad solamente responde a eventos externos (es reactiva).

Los posibles eventos son:

- La llegada de un mensaje.
- Activación del reloj.
- Un impulso espontáneo.

A excepción del impulso espontáneo, los eventos se generan bajo los términos del sistema.

Qué son las acciones

Secuencia finita y atómica de operaciones que realiza la entidad. Se ejecutan completamente o no se ejecutan.

Qué son las reglas

Las reglas indican cómo debe reaccionar una entidad ante un evento externo según su estado interno. Definen qué acción ejecutar en cada situación.

Qué es el comportamiento

El comportamiento es el conjunto de reglas que determinan cómo actúa una entidad frente a distintos eventos y estados.

Se lo llama también protocolo o algoritmo distribuido.

Qué restricciones siguen los mensajes

Los mensajes cumplen restricciones temporales:

- Siempre llegan (axioma).
- Tiene un retardo finito.
- Hay una orientación local, una entidad puede distinguir a quién enviar un mensaje y de quien recibe.
- Pueden haber delays:
    - Delay acotado: existe un delta, el cual, va a ser el límite de tiempo máximo que tardará un mensaje. Esto nos sirve para establecer timeouts.
    - Delay unitario: va a estar definida una cierta cantidad de tiempo como unidad de tiempo. Por ejemplo, el delay de un mensaje es una unidad de tiempo, puede representar 1s o 5s.

Esto permite tomar decisiones en algoritmos distribuidos.

Qué métricas se usan para comparar algoritmos distribuidos

Las medidas que se utilizan para la comparación de algoritmos son:

Cantidad de actividades de comunicación

- Cantidad de transmisiones o costo de mensajes.
- Carga de trabajo por entidad y carga de transmisión.

Tiempos

- Tiempo total de ejecución del protocolo.
- Tiempo ideal de ejecución: tiempo medido bajo ciertas restricciones, como delays de comunicación unitarios, relojes sincronizados, etc.

Qué tipos de eventos existen

Las entidades son reactivas y reaccionan a los eventos, estos pueden ser:

- Recepción de un mensaje.
- Impulso espontáneo.
- Alarma de reloj activada.

Los eventos desencadenan acciones a futuro. Los distintos delays resultan en distintas ejecuciones del protocolo con posibles resultados diferentes.

Qué es el estado interno de una entidad

Es la información que mantiene una entidad y que persiste en el tiempo. Determina cómo va a reaccionar ante los eventos. El estado de una entidad cambia con la ocurrencia de eventos.

Si definimos al estado interno de una entidad como Sigma en un tiempo t determinado, sea una entidad X que recibe el mismo evento en dos ejecuciones distintas, y Sigma1 y Sigma2 los estados internos, si Sigma1 = Sigma2 ⇒ el nuevo estado interno de X será el mismo en ambas ejecuciones.

Qué es el conocimiento de una entidad

Lo definimos con **Conocimiento local:** contenido de la memoria local de la entidad X y la información que se deriva.
En ausencia de fallas, el conocimiento no puede perderse.

Qué tipos de conocimiento de una entidad existen

Clasificamos al conocimiento en 3 tipos:

- **Información métrica:** toma mediciones sobre la red. Por ejemplo, la cantidad de nodos de un grafo, aristas, etc.
- **Propiedades topológicas:** conocimiento sobre la forma del grafo del ambiente distribuido, como están conectadas las entidades. Por ejemplo, si el grafo es un anillo, es acíclico, etc.
- **Mapas topológicos:** Es una representación de la topología del sistema distribuido. Muestra qué entidades están conectadas y cómo se comunican. Puede ser físico o lógico.

Final 2da fecha 2c2025

1) Modelo fork-join

A. Explicar qué características deben tener los problemas a modelizar para ser implementados utilizando el modelo fork-join.

B. Describir el algoritmo “robo de trabajo” (work stealing) que implementa el modelo fork-join.

C. Explicar y justificar por qué es más conveniente este, a tener una única cola de tareas.

Sol:

a) Para utilizarse el modelo de Fork-Join en un problema, este debe tener la característica de que se puede particionar en partes independientes una a la otra, con esto ganamos que se puedan ejecutar en paralelo a través de threads que al final, juntan los resultados formando el join. De esta manera minimizamos el tiempo de ejecución en comparación de hacerlo de manera secuencial.

b) En el modelo Fork-Join, cuando dividimos la tarea en subtareas independientes una a la otra, resolvemos separando la cantidad de tareas en partes iguales para cada thread. Cada thread va a tener una deque de dos extremos en donde, va a desencolar una tarea del “final”, la va a dividir en subtareas y las va a encolar en el final nuevamente, cada tarea que realice la va a tomar del final de la cola. Con esto, las tareas más antiguas serán las que están en el otro extremo, osea, al inicio de la cola. Cuando un thread se quede sin tareas en su cola, de forma aleatoria va seleccionar una cola de otro thread y le desencolara una tarea desde el “inicio” de la cola, siendo ésta la más antigua. Con esto minimizamos la posibilidad de tomar la misma tarea al mismo tiempo que el thread dueño de esa cola. Con este algoritmo, el balance de carga en cuanto a tareas queda equilibrado en algún momento y no quedan threads sin hacer nada.

c) Es más conveniente porque evita un cuello de botella central. Cada thread accede principalmente a su cola local, reduciendo sincronización y contención. Solo cuando se queda sin trabajo roba, lo que mejora la escalabilidad y balance de cargas.

2) Modelo de Actores

A. Explicar cuál es la motivación del modelo de actores.

B. Describir detalladamente las características de las entidades actor.

C. Describir el ciclo de vida de un actor en el framework Rust estudiado.

Sol:

a) La motivación del modelo de actores es la simplificación de la concurrencia, evitando los problemas clásicos como el estado mutable compartido, race condition y deadlocks.

En lugar de compartir memoria, los actores **se comunican exclusivamente por el pasaje de mensajes**, lo que permite diseñar sistemas distribuidos y concurrentes más seguros, escalables y fáciles de razonar. No necesita locks.

El modelo toma idea de la programación orientada a objetos (encapsulando su estado y comportamiento) y del pasaje de mensajes adaptándolo a entornos concurrentes y distribuidos.

b) El actor es la entidad fundamental de este modelo y tiene estas características:
- Encapsula estado interno y comportamiento.
- No hay estado mutable compartido.
- Se comunican únicamente por el pasaje de mensajes.
- Son ligeros.
- Tienen una dirección única.
- Tienen un mailbox (recurso FIFO para el recibo de mensajes).
- Procesan un mensaje a la vez.
- Jerarquía, un actor puede tener otros hijos actores.
- Mensajes inmutables.

c) El ciclo de vida de un actor pasa por 4 etapas:
- Primero se inicia con Started, donde se inicializa su estados internos y comportamientos. Listo para ejercer.
- Pasa a un estado de ejecución Running, en donde ya responde mensajes, realiza operaciones, etc.
- Un estado de “parando” Stopping, que se llama por stop en el mismo actor, en donde empieza a liberar espacio utilizado y recursos, siendo su estado siguiente el de parado.
- Por último, el estado “parado” Stopped, en donde ya libero la memoria y termina su ciclo de vida.

Más simple:
Started: Se inicia el estado interno.
Running: Procesa mensajes.
Stopping: Comienza a detenerse y liberar recursos.
Stopped: finaliza su ejecución.

3) Deadlocks

A. Explicar y detallar el algoritmo distribuido de detección de deadlocks en concurrencia distribuida.
B. Ejemplificar con un diagrama una posible ejecución que muestre la detección de un deadlock en el algoritmo anterior. Explicar cada paso.
C. Explicar qué se puede aplicar para evitar estas situaciones.

sol:
a) En el caso del distribuido, a diferencia del centralizado, no hay un líder que pueda manejar e ir generando un grafo con que proceso depende que otro proceso devuelve el recurso que necesita. Entonces, cada proceso se va a ir armando su grafo de dependencia

a medida que necesita recursos o se lo piden a él. El grafo se arma con nodos como procesos y aristas como dependencia de que otro proceso libere un recurso que necesita. Si un proceso A pide por un recurso que tiene otro proceso B y eventualmente recibe una petición del mismo proceso B por un recurso que tiene el proceso A, entonces se ve el grafo un ciclo, determinando que efectivamente hay una dependencia circular ⇒ deadlock.

b) Paso 1:
Proceso A tiene en lock recursos 2 y 4.
Proceso A necesita recurso 1.
Paso 2:
Pregunta a los demás procesos quien lo tiene.
Proceso B responde que lo tiene.
Paso 3:
Proceso A arma su grafo en donde depende del Proceso B.
[Proceso A] ⇒ [Proceso B]
Paso 4:
Proceso B necesita recurso 2, antes de devolver el recurso 1.
Pregunta a los demás procesos quien tiene el recurso 2.
Proceso A le responde al Proceso B que tiene el recurso.
Paso 5:
El Proceso A al recibir esa petición, agrega en su grafo otra arista.
[Proceso A] ⇒ [Proceso B]; [Proceso B] ⇒ [Proceso A]
Formándose el grafo cíclico y detectando el deadlock.

c) Hay dos algoritmos de prevención para no tener que llegar a esta situaciones.

wait-die: En este algoritmo, cada proceso al crear una transacción va a tener un **timestamp global** en donde, cuando el Proceso A necesite un recurso que tenga otro Proceso B, se va a comparar el **timestamp** de ambos y decidir de esta forma:
- **timestamp A < timestamp B** ⇒ Se espera a que termine de usar el recurso. El proceso A tiene timestamp más antiguo, por ende, es más viejo.
- **timestamp A > timestamp B** ⇒ Se aborta el Proceso A por ser más joven. Al ser más joven hay que deshacer menos modificaciones que para uno viejo que probablemente realizó más cambios.

wound-wait: En este algoritmo, misma situación que **wait-die** pero con distinto criterio para el **timestamp:**
- **timestamp A < timestamp B** ⇒ El Proceso A aborta el Proceso B porque el Proceso A es más viejo que el Proceso B y por ende, al abortar el proceso más joven, hay que rehacer menos modificaciones.
- **timestamp A > timestamp B** ⇒ El Proceso A es más joven entonces espera a que el Proceso B termine de utilizar el recurso.

Cuando queres menos **aborts** ⇒ **wait-die**.
Cuando queres menos **espera** ⇒ **wound-wait**.
Estos algoritmos evitan ciclos.

4) Ambientes distribuidos

A. **Explicar qué se entiende por “entidad” en un ambiente distribuido.**
B. **Enunciar las capacidades que tienen las entidades. Mencionar ejemplos.**
C. **Explicar qué se entiende por “restricciones temporales” en un ambiente distribuido.**

Sol:

a) En un ambiente distribuido, una entidad es la unidad de cómputo básica del sistema. Es un concepto flexible, ya que depende del marco de trabajo: una entidad puede ser un actor, un proceso, un thread, un nodo de red o incluso hasta una computadora completa.

Cada entidad:
- Tiene un estado interno que persiste en el tiempo.
- Ejecuta procesamiento local.
- Interactúa con otras entidades mediante eventos y mensajes.

Las entidades cooperan entre sí para cumplir el objetivo global del sistema distribuido.

b) Las entidades en un ambiente distribuido tienen las siguientes características:
- **Mantener estado interno:** variables internas, registros, reloj.
- **Procesar información localmente**: ejecutar un algoritmo, procesar un mensaje recibido.
- **Recibir eventos externos:** llegada de un mensaje, pedido de recurso, señal de red.
- **Ejecutar acciones atómicas:** actualizar estado interno, enviar un mensaje, cambiar comportamiento.
- **Comunicarse con otras entidades:** enviar y recibir mensajes a través de la red.

Estas capacidades permiten que las entidades reaccionen a los eventos y colaboren dentro del sistema distribuido.

c) Las restricciones temporales describen las condiciones bajo las cuales se transmiten y reciben los mensajes las entidades en el ambiente distribuido.
- **Axioma de llegada:** Todo mensaje que se envía, el algún momento llega.
- **Retardo finito:** Los mensajes que tardan en llegar, lo hacen en un tiempo finito (no es infinita la espera).
- **Delays acotados:** Es el tiempo máximo establecido para que un mensaje llegue a destino. Esto permite definir timeouts.
- **Delays unitarios:** El tiempo de transmisión de un mensaje es una unidad de tiempo (por ejemplo, puede ser 1s o 5s),

5) Transacciones / Diseño

Estamos diseñando un sistema de gestión de inventarios para una cadena de supermercados. Este sistema debe soportar transacciones concurrentes por múltiples usuarios que acceden a

inventarios en tiempo real. Las transacciones involucran operaciones de lectura y escritura (consultas y actualizaciones de stock por las ventas de productos).

El sistema enfrenta un alto nivel de contención de datos durante las horas pico, ya que muchas transacciones intentan acceder y modificar los mismos registros de inventario de forma simultánea.

a) Describir:
- Two Phase Locking (2PL)
- Timestamp
- Control de concurrencia optimista

b) Analizar ventajas y desventajas de cada uno en el contexto descrito.

c) Basándose en el análisis anterior, recomendar el algoritmo de control de concurrencia más adecuado. Justificar detalladamente teniendo en cuenta el nivel de concurrencia, la necesidad de mantener consistencia de datos y el impacto en el rendimiento del sistema durante las horas pico.

Sol:
a) Los algoritmos de transacciones que piden describir son los siguientes:

Two Phase Locking: Como lo dice el nombre, consiste en dos fases de lock:
- **Fase de crecimiento:** Es la etapa en la que el proceso al iniciar la transacción, empieza a pedir locks, puede realizar operaciones entre cada obtención de recursos pero generalmente pide los recursos a utilizar y luego realiza todas las operaciones necesarias con ellos.
- **Fase de decrecimiento:** El proceso cuando termina de realizar las operaciones, empieza a devolver los locks de los recursos que previamente había solicitado, es entonces cuando entra en esta etapa de decrecimiento y no debe pedir ningún lock de algún recurso nuevamente. Es posible seguir realizando operaciones sobre los recursos que mantiene el lock cuando entra en esta etapa pero, generalmente devuelve todos los recursos luego de realizar todas sus operaciones en total.

Timestamp: Se establecen **timestamps** de lectura y escritura para TODOS, los procesos al iniciar una transacción y a los propios recursos. Entonces cuando un proceso quiere acceder a un recurso, corrobora el timestamp de dicha acción. Si su timestamp es mayor que el del archivo, entonces realiza la acción, en caso contrario aborta la transacción.

Concurrencia Optimista: Cada proceso al iniciar una transacción realiza una copia en su memoria, realiza todas sus operaciones y al finalizar, corrobora que ningún otro proceso haya hecho una transacción modificando los mismos recursos. En caso de que ningún proceso haya utilizado los mismos recursos, establece los cambios en el storage, en caso contrario simplemente descarta los cambios.

b)

Two Phase Locking:

- **Ventajas:** Cada vendedor toma primero locks de los productos (recursos) que necesita vender, siendo estos los pedidos por el cliente, antes de realizar la venta, garantizando al cliente que tiene todos los productos en su totalidad.
- **Desventajas:** Puede pasar que haya un deadlock entre vendedores que quieran los mismos productos.

Timestamp:

- Ventajas: No van a haber deadlocks porque no hay locks.
- Desventajas: Pueden haber muchos abort/retries y posible starvation en hora pico.

Concurrencia optimista:

- Ventajas: Excelente cuando hay pocos conflictos (pocas ventas) y muchas lecturas de productos.
- **Desventajas:** En hora pico pueden haber muchísimos aborts, con lo cual hay mucho desperdicio de trabajo, baja performance.

c) Para este tipo de problema, la mejor opción es **Two Phase Locking** con algún algoritmo de prevención de deadlock como wait-die o would-wait. Ya que es preferible bloquear y esperar en caso de un conflicto, antes que abortar muchas veces en hora pico, con esto minimizamos la cantidad de aborts y tenemos los datos actualizados constantemente, tiene desventajas como mantener una sincronización constante pero resuelve con alta performance.

Concurrencia Optimista sería mejor para un escenario en donde casi no hay conflictos. Por ejemplo, que fuera un sistema con muchas consultas y pocas escrituras como un catálogo online, portal de precios, sistemas de reportes, donde muchos lees y pocos escriben.

Timestamps conviene cuando quiero evitar locks y deadlocks, pero acepto aborts moderados. Por ejemplo, un sistema distribuido con alta latencia como sistemas entre sucursales, réplicas geográficas y cloud distribuido. Conviene en sistemas distribuidos donde los locks son costosos.

Two Phase Locking conviene cuando hay muchos conflictos.

Final 3ra Fecha 2c2025

Ejercicio 1 - Monitores

a) Explicar y describir qué es un monitor y qué métodos implementa.

b) Enumerar las similitudes y diferencias que se pueden encontrar entre un semáforo y un monitor.

c) Describir un ejemplo de situación en la que sea conveniente utilizar un monitor en lugar de un semáforo. Justificar la elección.

Sol:

a) Un monitor es una herramienta de sincronización que garantiza la exclusión mutua entre procesos y permite que los procesos se bloqueen hasta que se complete una condición.
Solo un proceso puede ejecutar el monitor a la vez.
El monitor utiliza variables de condición, que no almacenan valores y tienen una cola FIFO de procesos bloqueados esperando entrar.
Tiene 3 métodos, **waitC**, que bloquea el proceso y libera el monitor, **signalC**, que despierta a un proceso bloqueado, y **empty**, que indica si hay procesos esperando.

b) Tanto semáforos como monitores son herramientas de sincronización que permiten coordinar procesos concurrentes y serializar la entrada a recursos compartidos, evitando race condition.
Sin embargo, presentan diferencias en sus métodos: **wait** en semáforos bloquea al proceso solo si el contador del semáforo es igual a 0, **waitC** del monitor bloquea siempre. **Signal** del semáforo permite que otro proceso obtenga el acceso o que simplemente modifique el contador, mientras que **signalC** del monitor puede no hacer nada si la cola FIFO está vacía.
Además, en semáforos, cuando se elige el siguiente proceso bloqueado se hace de manera al azar, mientras que en monitores es por orden de llegada.
Los monitores encapsulan los datos y el protocolo de acceso, reduciendo errores de programación, mientras que los semáforos no ofrecen encapsulamiento.

c) Una situación en la que conviene más utilizar un monitor en vez de semáforos, es en una oficina en la que haya solamente una impresora. Solamente queremos que un proceso ejecute a la vez la impresión, siendo esta una única condición a cumplirse y con un protocolo en específico, no queremos delegar ese protocolo a un proceso y que haga lo que quiera. Con un monitor nos aseguramos encapsulamiento por parte del comportamiento de la impresora, sincronización y orden por la cola FIFO.

Ejercicio 2 - Programación Asincrónica

a) Explicar qué se entiende por Programación Asincrónica.

b) Comparar el modelo de Programación Asincrónica con el modelo tradicional de threads o de procesos del Sistema Operativo. Describir ventajas y desventajas.

c) Describir un ejemplo de caso de uso en el que convenga utilizar el modelo asincrónico y otro ejemplo para el modelo de threads. Justificar cada elección.

Sol:
a) El modelo de programación asincrónica es un paradigma de programación que se utiliza para tareas livianas y de mucha espera, como lo son las operaciones de I/O. Utilizando pocos threads y aprovechando ese tiempo de espera. Al momento de ejecutar una operación de I/O hasta que llegue la respuesta, devuelve el control al executer para que siga con otra tarea que no dependa de esta respuesta.

b) El modelo de programación asincrónica utiliza menos recursos, menos threads y aprovecha el tiempo ocioso de espera por una operación de I/O, solamente es productivo cuando se trata de tareas livianas, mientras que el modelo tradicional de threads en el caso tener una operación de I/O se bloquea esperando la respuesta pero, este modelo se utiliza para otros casos en lo que sean tareas de alto cómputo, aprovechando al máximo el CPU mediante varios threads.

c) La programación asincrónica es conveniente en un servidor web, donde se manejan muchas peticiones concurrentes que realizan operaciones de red o base de datos, permitiendo atender miles de conexiones con pocos threads.

El modelo de threads es más adecuado para tareas CPU-bound, como el procesamiento o generación de imágenes, donde se busca aprovechar múltiples núcleos de CPU en paralelo.

Ejercicio 3 - Deadlocks

a) ¿Qué se entiende por deadlock? ¿Qué dificultades adicionales presenta la posibilidad de existencia de deadlocks en un ambiente de Concurrencia Distribuida?

b) Describir y comparar los algoritmos de detección de Deadlock Centralizado y Distribuido.

Explicar claramente cómo se determina la existencia de un deadlock en cada caso.

c) Analizar las ventajas y desventajas de cada algoritmo. ¿En qué escenarios conviene utilizar cada uno?

Sol:
a) Cuando dos procesos necesitan bloquear un recurso para continuar que cada uno tiene del otro, esto es una espera circular que nunca va a terminar ⇒ deadlock.
En ambientes de concurrencia distribuida es un poco más complejo detectarlos que en un ambiente centralizado en una computadora, por problemas de red (latencias de mensajes, desconexión, etc) pueden generar falsos deadlocks.

b) El algoritmo centralizado de detección de deadlocks, en este ya estamos en un ambiente distribuido centralizado en el que existe un líder, es quien conoce y a quien los procesos preguntan para la obtención de un recurso, entonces el líder va a ir armando un grafo con nodos como procesos y aristas conectadas por los procesos que dependan de un recurso que tiene otro proceso. Si se forma un ciclo, es decir, un grafo cíclico, entonces hay una dependencia circular, y por ende, un deadlock.
El algoritmo distribuido de detección de deadlocks, a diferencia del centralizado, cada proceso debe consultar a todos por un recurso en específico, entonces cada proceso se va a ir armando su propio grafo a medida que pide un recurso o se lo solicitan. Si solicita un recurso a un proceso, queda esperando y eventualmente le llega una petición de ese mismo proceso solicitando un recurso que tiene el, entonces es una dependencia circular, y por ende, un deadlock.

c) Las ventajas del centralizado es que solamente un proceso debe armar el grafo reduciendo complejidad pero, es un único punto de falla, siendo este un cuello de botella. Las ventajas para el distribuido es más de un punto de falla, dando más garantía ante alguna caída pero, como desventaja es más complejo que todos los nodos tengan que mantener su grafo actualizado y verificando ciclos.

El centralizado conviene más cuando se habla de un datacenter con buena conectividad a red y alta performance de máquina, mientras que el distribuido conviene más cuando hablamos de un ambiente distribuido con floja conectividad y objetivos simples de hacer por cada entidad que lo compone.

Ejercicio 4 - Elección de Líder y Ambientes Distribuidos
a) Describir el Algoritmo Ring para la elección de líder en un Ambiente Distribuido.
b) Realizar un diagrama con un ejemplo de un posible escenario, que muestre todos los pasos de la ejecución del algoritmo.
c) Enumerar los tipos de eventos que se consideran en un Ambiente Distribuido.
d) Explicar qué es la Acción, Regla y Comportamiento de una entidad en un Ambiente Distribuido.

Sol:
c) Las entidades de un ambiente distribuido son reactivas, reaccionan frente a eventos.
- Recepción de un mensaje.
- Impulso espontáneo.
- Alarma de reloj.

d) Acción es el conjunto finito y atómico de operaciones que realiza una entidad. Se ejecutan todas o no se ejecutan.
Regla es como debe reaccionar una entidad frente a un evento externo dependiendo de su estado interno. Determinan qué acción ejecutar.

Comportamiento es el conjunto de reglas que determinan cómo actúa una entidad frente a un evento externo o un estado interno. Se llama también protocolo o algoritmo distribuido.

Ejercicio 5 - Redes de Petri
Sistema de Carga de Vehículos Eléctricos

Una estación de carga de vehículos eléctricos opera bajo las siguientes reglas:
1. Hasta 2 vehículos pueden cargarse simultáneamente en la estación.
2. Si la estación ha completado 4 cargas seguidas (es decir, 4 procesos de carga finalizados), un técnico debe realizar un mantenimiento preventivo antes de permitir nuevas cargas.
3. El mantenimiento sólo puede realizarse si ningún vehículo está cargando en la estación.
4. Las acciones de "iniciar carga", "finalizar carga" y "realizar mantenimiento" son atómicas.

Consignas:
5.1) Modelar este sistema con una Red de Petri, incluyendo:
*** Lugares (con marcas iniciales). Explicar claramente el significado de cada uno de los lugares.**
*** Transiciones y arcos (usar arcos inhibidores si es necesario).**
*** Anotaciones claras (ej: "Iniciar carga", "Realizar mantenimiento").**
5.2) Mostrar un grafo de alcance sobre una posible ejecución.
5.3) Explicar cómo el modelo desarrollado garantiza que el mantenimiento solo se realiza cuando no hay vehículos cargando.